AN INTERNSHIP PROJECT REPORT

ON

“MATHME 3D STUDIO”

Generative 3D Art & Object Studio

Submitted in partial fulfilment of the requirements for the award of the degree of

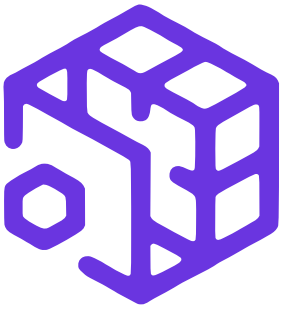
BACHELOR OF COMPUTER APPLICATIONS (BCA)

Semester III

Submitted by

SULAIMAAN

Register Number: _____



Under the guidance of

_____

Designation: _____

DEPARTMENT OF COMPUTER APPLICATIONS

[Name of the College] _____

[Affiliated University] _____

Academic Year 2026 – 2027

INTERNSHIP CERTIFICATE

This is to certify that **SULAIMAAN**, Register Number _____, a student of the Third Semester of the Bachelor of Computer Applications (BCA) programme, has successfully completed the internship project titled **"MATHME 3D STUDIO: Generative 3D Art & Object Studio"** as an independent internship project.

The internship was carried out on a part-time basis for a total of **90 hours** over **10 weeks**, from _____ to _____. During this period the student analysed, designed, developed, tested and documented the MathMe web application described in this report.

The work presented in this report is a record of the work done by the student, and the conduct of the student during the internship was found to be satisfactory.

Place: _____

Date: _____

Signature of the Internship Guide / Mentor	Seal
Name: _____	

[NAME OF THE COLLEGE]

[Address of the College] _____

DEPARTMENT OF COMPUTER APPLICATIONS

CERTIFICATE

This is to certify that the internship project report entitled **"MATHME 3D STUDIO: Generative 3D Art & Object Studio"** is a bonafide record of the internship work carried out by **SULAIMAAN**, Register Number _____, in partial fulfilment of the requirements for the Third Semester of the **Bachelor of Computer Applications (BCA)** degree of _____ during the academic year **2026 – 2027**.

It is certified that all corrections and suggestions indicated during the internal assessment have been incorporated in the report. The report satisfies the academic requirements prescribed for the internship.

Signature of the Guide	Signature of the Head of the Department
Name: _____	Name: _____

Signature of the Principal

Name: _____

Examiners

1. Examiner	2. Examiner
Name: _____	Name: _____

STUDENT DECLARATION

I, **SULAIMAAN**, Register Number _____, a student of the Third Semester of the Bachelor of Computer Applications (BCA) programme, hereby declare that the internship project report entitled **"MATHME 3D STUDIO: Generative 3D Art & Object Studio"** is an original record of the work carried out by me as an independent internship project, under the guidance of _____.

I further declare that this report is based on my own work, that it has not been submitted earlier to any other university or institution for the award of any degree, diploma or certificate, and that all sources of information used in this report have been duly acknowledged in the bibliography.

Place: _____

Date: _____

_____

(SULAIMAAN)

ACKNOWLEDGEMENT

The successful completion of this internship project would not have been possible without the guidance, support and encouragement of many people. I take this opportunity to express my sincere gratitude to all of them.

I express my deep sense of gratitude to _____, Principal, _____, for providing the necessary facilities and an encouraging environment to carry out this internship.

I am thankful to _____, Head of the Department of Computer Applications, for the constant support and for giving me the opportunity to take up this internship project.

I owe my sincere thanks to my internship guide, _____, for the valuable guidance, timely suggestions and patient reviews at every stage of this work, from the first idea to the final report.

I also thank all the teaching and non-teaching staff of the Department of Computer Applications for their help and cooperation throughout the internship period.

I am grateful to the open-source communities behind React, Three.js, Mantine, FastAPI, trimesh and manifold3d, whose tools and documentation made this project possible, and to the HuBMAP consortium for making the Human Reference Atlas heart model freely available for education.

Finally, I thank my parents, family members and friends for their love, patience and constant encouragement, which kept me motivated throughout this internship.

(SULAIMAAN)

TABLE OF CONTENTS

	Page
EXECUTIVE SUMMARY	**10**
CHAPTER I: INTRODUCTION	**11**
1.1 Background of the Internship Project	11
1.2 Problem Statement	11
1.3 Existing System and Proposed System	12
1.4 Objectives of the Project	13
1.5 Scope of the Project	14
1.6 Methodology	14
1.7 Internship Details	15
1.8 Tools and Technologies Used	15
1.9 Organisation of the Report	16
CHAPTER II: DESCRIPTION OF THE ORGANIZATION	**17**
2.1 About MathMe	17
2.2 Vision and Mission	17
2.3 Target Users	18
2.4 Products and Services	18
2.5 Business Model and Plans	19
2.6 Project Team and Roles	20
2.7 Working Environment and Practices	20
CHAPTER III: EXPERIENTIAL LEARNING	**23**
3.1 Weekly Work Log	23
3.2 Requirement Analysis	24
3.3 System Design	27
3.4 Implementation	32
3.5 Output Screens	41
3.6 Testing	51
3.7 Deployment Plan	52
3.8 Security Measures	53
3.9 Learning During the Internship	54
CHAPTER IV: INTERNSHIP OUTCOMES AND CONCLUSION	**55**

4.1 Internship Outcomes 55
4.2 Achievements in Numbers 55
4.3 Skills Gained 56
4.4 Challenges Faced and Solutions 57
4.5 Limitations 57
4.6 Future Enhancements 58
4.7 Conclusion 58
BIBLIOGRAPHY **59**

LIST OF FIGURES

Page

Figure 2.1: Landing Page 22

Figure 2.2: Pricing Section of the Landing Page 22

Figure 3.1: System architecture of MathMe 27

Figure 3.2: Context diagram (Level 0 DFD) 28

Figure 3.3: Level 1 data flow diagram 29

Figure 3.4: Entity-relationship diagram of the account database 30

Figure 3.5: Use case diagram 31

Figure 3.6: Module structure of the MathMe repository 32

Figure 3.7: Sequence diagram of a Plus checkout 39

Figure 3.8: Home Page 42

Figure 3.9: 3D Studio Page (100 Spheres in a Spiral) 42

Figure 3.10: Pattern Panel 43

Figure 3.11: Learn Panel 43

Figure 3.12: Measure Panel 44

Figure 3.13: Ideas Gallery 44

Figure 3.14: Export Page 45

Figure 3.15: Heart Cut Page (Four-Chamber View) 45

Figure 3.16: Pencil Drawing Page 46

Figure 3.17: 2D Sketch Board 46

Figure 3.18: Upgrade Prompt 47

Figure 3.19: Plans Page 47

Figure 3.20: Checkout Page 48

Figure 3.21: Account Settings Page 48

Figure 3.22: Class Page 49

Figure 3.23: Owner Dashboard 49

Figure 3.24: Campus Licences Page 50

Figure 3.25: Revenue Calculator Page 50

LIST OF TABLES

Page

Table 1.1: Existing systems compared with the proposed system 12

Table 1.2: Internship details 15

Table 1.3: Tools and technologies used 15

Table 2.1: MathMe modules and services 18

Table 2.2: MathMe plans (prices before 18% GST) 19

Table 2.3: Roles in the internship project 20

Table 3.1: Weekly work log 23

Table 3.2: Functional requirements 24

Table 3.3: Non-functional requirements 25

Table 3.4: Hardware and software requirements 26

Table 3.5: Patterns and their formulas 33

Table 3.6: Export formats 35

Table 3.7: Prices with 18% GST 38

Table 3.8: Selected test cases 51

Table 3.9: Summary of automated test results 52

Table 3.10: Deployment steps 53

Table 4.1: The project in numbers 55

Table 4.2: Skills gained during the internship 56

Table 4.3: Challenges and how they were solved 57

EXECUTIVE SUMMARY

This report presents the internship project **"MathMe 3D Studio: Generative 3D Art & Object Studio"**, carried out as an independent internship project during the Third Semester of the Bachelor of Computer Applications (BCA) programme. The internship comprised **90 hours** of part-time work spread over **10 weeks**.

Traditional 3D modelling software asks the user to create, move, rotate and resize every object by hand. Placing two hundred spheres in a spiral can take hours, and beginners must learn a large number of tools before they make anything. At the same time, school students often see mathematics (angles, sine and cosine, sequences, areas and volumes) as abstract formulas with no visible use. MathMe was built to connect these two problems: **the student writes one mathematical rule, and the computer places every object.**

MathMe is a web application that runs in any modern browser. The user picks a shape by its width, height and radius, chooses one of seven patterns (spiral, grid, circle, wave, radial rings, sphere and random scatter) and sees hundreds or thousands of copies appear instantly in a 3D view. A "Learn" panel shows the formula behind every pattern with the user's own numbers filled in, and a "Measure" panel works out volume and surface area step by step. Finished work can be exported as GLB, STL, OBJ, PNG or PDF files for 3D printing, games, posters and class reports.

During the internship the application grew into a complete product: a **maths engine** in TypeScript; a **3D studio** built with React and Three.js, with a pencil tool and a 2D sketch board; a **geometry service** in Python (FastAPI, trimesh, manifold3d) for exact measurements, joining and cutting solids and a human-heart slice atlas; and an **account service** for Cloudflare Workers with Google sign-in, cloud saves, plans in Indian rupees with 18% GST, Cashfree payments, school licences and an owner dashboard.

The work followed an iterative method of small, tested milestones. The finished system passes **273 automated tests**, run on every change. The internship gave practical experience of the full software development life cycle, from requirements and design to programming, databases, payments, security, testing and documentation.

CHAPTER I: INTRODUCTION

1.1 Background of the Internship Project

Computer graphics and 3D printing are becoming part of school and college laboratories, and students are increasingly asked to design models, posters and simulations. Most 3D tools, however, are built for professional artists. They expose hundreds of buttons, and they treat every object as a separate item to be placed by hand.

Mathematics offers a better way to describe many designs. A sunflower, a spiral staircase, a DNA double helix, a snowflake or a field of waves can each be described by a short rule: how many objects, which pattern, how far apart, and how much each one turns or grows. If a computer program understands such rules, a student can create a complex artwork in seconds and, at the same time, see exactly how angles, trigonometry and sequences produce the shapes in front of them.

The project brief, titled "Generative 3D Art & Object Studio", asked for exactly this kind of tool. Its example rule, **"100 objects → Spiral → Radius 20 → Rotation 30° → Scale 0.5–2"**, became the guiding test case of the internship. MathMe 3D Studio is the result: a browser-based studio, written with 10th-grade students in mind, where mathematics is the main design tool.

1.2 Problem Statement

The internship set out to address the following problems:

- **Repetitive manual modelling.** Creating patterned 3D artwork in traditional software means creating, moving, rotating and resizing each object separately, which is slow and error-prone.
- **Steep learning curve.** Beginners must learn a large set of tools and technical vocabulary (vertices, modifiers, transforms) before they can make anything.
- **Mathematics without context.** Students learn formulas such as $x = r \cos \theta$ or the volume of a sphere, but rarely see them used to build something real.
- **Hard-to-share results.** Many tools produce files that only the same tool can open, which makes printing, presenting and sharing difficult in a classroom.

- **Sustainability.** A free educational tool also needs a fair, legal way to cover its hosting costs in India (plans, GST invoices and school licences).

The aim was therefore to build a tool in which one mathematical rule creates every object, every formula is visible and explained, and the result can be exported to standard file formats, while keeping the interface simple enough for a school student.

1.3 Existing System and Proposed System

Before designing MathMe, the tools that students and teachers already use were studied. Each is good at its own job, but none of them combines pattern-based 3D modelling, visible mathematics and classroom-friendly exports in one simple tool.

- **Professional 3D software** (for example Blender) is very powerful and can make repeated patterns with array modifiers or node systems, but it is designed for trained artists. A beginner must learn many concepts before making a simple pattern, and the mathematics stays hidden inside the tool.
- **Beginner 3D design tools** (for example Tinkercad) are easy to start with and export files for 3D printing, but objects are still placed one at a time, and the tools do not explain the mathematics behind a design.
- **Mathematics software** (for example GeoGebra 3D) shows graphs, surfaces and formulas very well, but it is not built for making artwork from hundreds of objects or for producing posters and project sheets.

Table 1.1: Existing systems compared with the proposed system

Feature	Professional 3D tools	Beginner 3D tools	Maths software	MathMe (proposed)
Easy for a 10th-grade student	No	Yes	Partly	Yes
One rule places many objects	Yes, with advanced tools	No	Partly	Yes, 7 patterns
Shows the formula for every object	No	No	Yes	Yes, with worked examples

Feature	Professional 3D tools	Beginner 3D tools	Maths software	MathMe (proposed)
Volume and area with working	No	No	Partly	Yes
Plain-language recipe	No	No	No	Yes
3D-print and game exports	Yes	Yes	Partly	Yes
Runs in a browser, no installation	No	Yes	Yes	Yes
Class page for teachers	No	Yes	Yes	Yes

The **proposed system**, MathMe, keeps the ease of beginner tools and the clear formulas of mathematics software, and adds the pattern power of professional tools: the user writes one rule, sees every object placed by it, reads the formula behind it, and exports real files.

1.4 Objectives of the Project

The main objectives of the internship project were:

- To design and develop a web-based 3D studio in which shapes are described by simple measurements (width, height, depth, radius).
- To implement a generation engine with seven mathematical patterns that place hundreds or thousands of copies of a shape automatically.
- To show the formula behind every pattern and shape, worked out with the user's own numbers, so that the tool also teaches mathematics.
- To support a plain-language "recipe" such as "100 spheres → spiral → radius 20", so that a model can be described in one line.
- To export the result as GLB, STL, OBJ, PNG and PDF files for games, 3D printing, posters and reports.
- To build a Python geometry service for exact measurements, joining and cutting of solids, and an educational human-heart slice atlas.
- To add a 2D sketch board with points, lines, circles, measurements and geometric rules, and to turn sketches into 3D solids.

- To add user accounts, cloud saves, subscription plans with GST-compliant invoices, school licences and an owner dashboard, so that the product can be offered sustainably.
- To test every part automatically and document the system for future maintenance and deployment.

1.5 Scope of the Project

In scope: the complete web application (home page, 3D studio, 2D sketch board, exports, settings and help), the geometry and account services, the landing page with pricing, draft policy pages, automated tests and the deployment guide. The application works fully without an account; signing in adds cloud saves, paid plans and school features.

Out of scope (left for later phases): native mobile applications, real-time collaborative editing, class assignments and grading, a marketplace and a public developer API. The heart atlas is for education only and is not a medical record.

1.6 Methodology

The internship followed an **iterative and incremental** development method, similar to the agile practice of short sprints. The work was divided into milestones; each milestone was planned, implemented, tested and committed to the version-control repository before the next one began. This kept the application working at every stage and made it easy to show progress to the guide.

Every milestone followed the same cycle:

- **Plan:** write down what the milestone must achieve and how it will be tested.
- **Design:** sketch the data model, the user interface and the interfaces between parts.
- **Implement:** write the code in small, readable steps.
- **Test:** write unit tests for the logic and browser tests for the user flows; run lint, formatting and type checks.
- **Review and commit:** read the change again, fix problems, and commit it with a clear message.

The milestones were: project set-up; the maths engine and 3D view; custom shapes; recipes and project files; exports; the geometry service; the heart atlas; interface design and the home page; the pencil and the 2D sketch board; and finally accounts, plans, payments, school

licences and the owner dashboard. Table 3.1 in Chapter III maps them to the ten weeks of the internship.

1.7 Internship Details

Table 1.2: Internship details

Particular	Details
Name of the student	Sulaimaan
Register number	_____
Programme and semester	Bachelor of Computer Applications (BCA), Semester III
Title of the internship project	MathMe 3D Studio: Generative 3D Art & Object Studio
Type of internship	Independent internship project (software development)
Mode	Part-time, during the semester
Duration	90 hours over 10 weeks
Period	From _____ to _____
Internal guide	_____
Domain	Educational technology, 3D graphics, web development

1.8 Tools and Technologies Used

The project uses free and open-source tools throughout. Table 1.3 lists the main technologies and the purpose of each one.

Table 1.3: Tools and technologies used

Area	Technology	Purpose in MathMe
Front-end language	TypeScript 5	Type-safe code for the whole web application
User interface	React 19, Mantine 9	Components, panels, dialogs and forms
3D graphics	Three.js, react-three-fiber, drei	3D view, instanced rendering, cameras and gizmos
State	zustand, immer, zundo	Application state with undo and redo
Build tool	Vite 8	Fast development server and production build

Area	Technology	Purpose in MathMe
Back-end language	Python 3.11+	Geometry service and account service
Web framework	FastAPI, Pydantic	REST APIs with validated input
Geometry	trimesh, manifold3d	Exact volumes, joining and cutting solids
Hosting (planned)	Cloudflare Workers, D1, R2	Serverless API, SQLite database, file storage
Payments	Cashfree Payments	UPI AutoPay subscriptions, orders, payment links
Email	Resend	Receipts and renewal reminders
Sign-in	Google OAuth 2.0	Secure sign-in without passwords
Documents	jsPDF	PDF project sheets and GST invoices in the browser
Testing	Vitest, Playwright, pytest	Unit tests, browser tests, service tests
Quality	ESLint, Prettier, ruff	Linting and consistent formatting
Version control	Git, GitHub Actions	History, branches and continuous integration
Editor	Visual Studio Code	Writing and debugging code

1.9 Organisation of the Report

Chapter II describes the project setting: MathMe's vision, users, products, business model and way of working. **Chapter III** describes the experiential learning: the weekly work log, requirements, system design, implementation, output screens, testing and what was learnt. **Chapter IV** summarises the outcomes, skills, challenges, limitations and future enhancements. The **Bibliography** lists the books and documentation used.

CHAPTER II: DESCRIPTION OF THE ORGANIZATION

This internship was carried out as an **independent internship project** and not inside an outside company. In place of a company profile, this chapter describes the project itself as the organisation in which the work was done: its vision and mission, its users, the products it offers, its business model, and the way the work was planned and managed.

2.1 About MathMe

MathMe 3D Studio is a generative 3D art and object studio for students, teachers and makers. Its name combines “Math” and “Me”: mathematics made personal, visible and creative. The product runs entirely in a web browser, so it needs no installation and works on school computers, laptops and tablets.

The central idea of MathMe is that **one rule creates every object**. Instead of placing two hundred spheres by hand, the user writes a rule (how many, which pattern, which sizes) and the studio calculates every position, rotation and scale. Changing one number updates the whole artwork. Every rule explains itself in a Learn panel, so the same tool is both a creative studio and a mathematics lesson.

The public face of MathMe is its landing page. It and its pricing section are shown in Figures 2.1 and 2.2 at the end of this chapter.

2.2 Vision and Mission

Vision. To make mathematics the most creative subject in school by letting every student build real 3D art and objects from simple rules.

Mission.

- To give students a free, friendly studio in which every formula is visible and every shape is described in plain words.
- To let teachers use the same tool for lessons on geometry, trigonometry, sequences, area and volume, and for science topics such as human anatomy.
- To produce real files (3D prints, models for games, posters and reports) so that students can take their work beyond the screen.
- To keep the product sustainable through fair, transparent pricing in Indian rupees and affordable school licences.

2.3 Target Users

- **School students (Classes 8–12):** learn geometry and trigonometry by building artwork; the interface was written with 10th-grade students in mind.
- **College students:** use MathMe for design, engineering-drawing and computer-graphics coursework, and for 3D printing.
- **Teachers:** demonstrate formulas live, set creative tasks and see their students' shared work on a class page.
- **Makers and designers:** create patterned objects, print-ready models and posters quickly.
- **Schools and colleges:** buy a Campus licence for a whole class with teacher and student accounts.

2.4 Products and Services

MathMe is organised as a set of modules that work together. Table 2.1 summarises each module and what it offers to the user.

Table 2.1: MathMe modules and services

Module	What it offers
Home page	Project list with pictures, an AI-assisted chat box ("describe what to make"), an ideas gallery
3D studio	21 shape types, 7 patterns, variation, symmetry, colours, undo/redo, a plain-language recipe box
Learn and Measure	The formula behind every pattern with a worked example; volume and surface area with full working
Pencil	Draw closed outlines that become solids, or 3D pen lines that become tubes, directly in the 3D view
2D sketch board	Points, lines and circles by radius, snapping, measurements, geometric rules, 2D-to-3D
Exports	GLB, STL, OBJ, PNG (HD, 4K, square, see-through), PDF project sheet, SVG, DXF, project file
Geometry service	Exact volume of the whole model, join / cut / overlap of solids, print-ready joined STL

Module	What it offers
Heart atlas	A real human heart (Human Reference Atlas) with 51 structures, cut along 21 standard planes or any angle
Accounts and cloud	Google sign-in, cloud saves, plans and limits, GST invoices
Campus	School licences, join codes, class page with shared student projects
Owner dashboard	Sign-ups, active users, paying users, MRR, churn, enquiries, licences, revenue calculator

2.5 Business Model and Plans

MathMe follows a **"freemium"** business model. Everything can be tried for free, and paid plans add more capacity, more export formats and commercial-use rights. All prices and limits are stored in one file (shared/plans.json), which is read by the web application, the server and the tests, so that the prices shown can never disagree with the prices charged. Prices are in Indian rupees and **18% GST is added at checkout**: CGST and SGST (9% each) within the seller's state, or IGST (18%) for other states.

Table 2.2: MathMe plans (prices before 18% GST)

Plan	Price	Main features
Free (Explorer)	₹0	3 patterns, 2,000 objects per scene, 3 cloud projects, 15 exports a month (PNG HD, project file), 5 AI messages and 5 geometry jobs a day
Plus (Creator)	₹299 a month or ₹2,499 a year	All 7 patterns, 20,000 objects, 2D-to-3D, GLB/STL/OBJ/SVG/DXF/PDF, 200 exports a month, 50 cloud projects, 50 AI messages and 50 geometry jobs a day
Pro (Professional)	₹799 a month or ₹6,999 a year	4K and see-through PNG, print-ready STL, 1,000 exports a month, 500 cloud projects, 300 geometry jobs a day processed first, commercial use, priority support
Campus	₹14,999 a year	100 student accounts (Plus) and 5 teacher accounts (Pro), class page and join codes; starts with a free pilot

Plan	Price	Main features
Enterprise	By quotation	Volume licences and custom workflows for universities, laboratories and studios

For example, the Plus plan costs ₹299 + ₹53.82 GST = **₹352.82** a month, and the Campus licence costs ₹14,999 + ₹2,699.82 GST = **₹17,698.82** a year. Monthly plans renew through a UPI AutoPay or card mandate; yearly plans are a single payment with reminder emails before they end.

2.6 Project Team and Roles

Because this was an independent internship project, the student carried out every role in the project, under the supervision of the internal guide. Table 2.3 lists the roles.

Table 2.3: Roles in the internship project

Role	Person	Responsibilities
Developer and designer	Sulaimaan	Requirements, design, front-end and back-end programming, user interface, diagrams
Tester	Sulaimaan	Unit tests, browser tests, service tests, manual checks on desktop and phone sizes
Documentation	Sulaimaan	Plan, README, deployment guide, policy drafts and this report
Internal guide	_____	Reviews, suggestions and evaluation of the internship

2.7 Working Environment and Practices

The internship used a professional software engineering environment, so that the project could be maintained and extended after the internship:

- **Version control:** all code lives in a Git repository on GitHub. Work is done on a feature branch, and every milestone is a separate commit with a descriptive message.
- **Monorepo structure:** the web application (web/), the geometry service (geometry-service/), the account service (account-service/), shared data (shared/) and documentation

(docs/) are kept in one repository.

- **Continuous integration:** a GitHub Actions pipeline runs linting, formatting checks, type checking, unit tests, the production build, browser tests and the Python tests on every push.
- **Code quality:** ESLint and Prettier for TypeScript, ruff for Python; strict TypeScript settings; small, readable functions with comments where the reason is not obvious.
- **Documentation:** a project plan (docs/PLAN.md), a README with setup and checks, and a step-by-step deployment guide (docs/DEPLOY.md).
- **Working hours:** about nine hours a week on a part-time basis, with a short written log of the work done each week (Table 3.1).

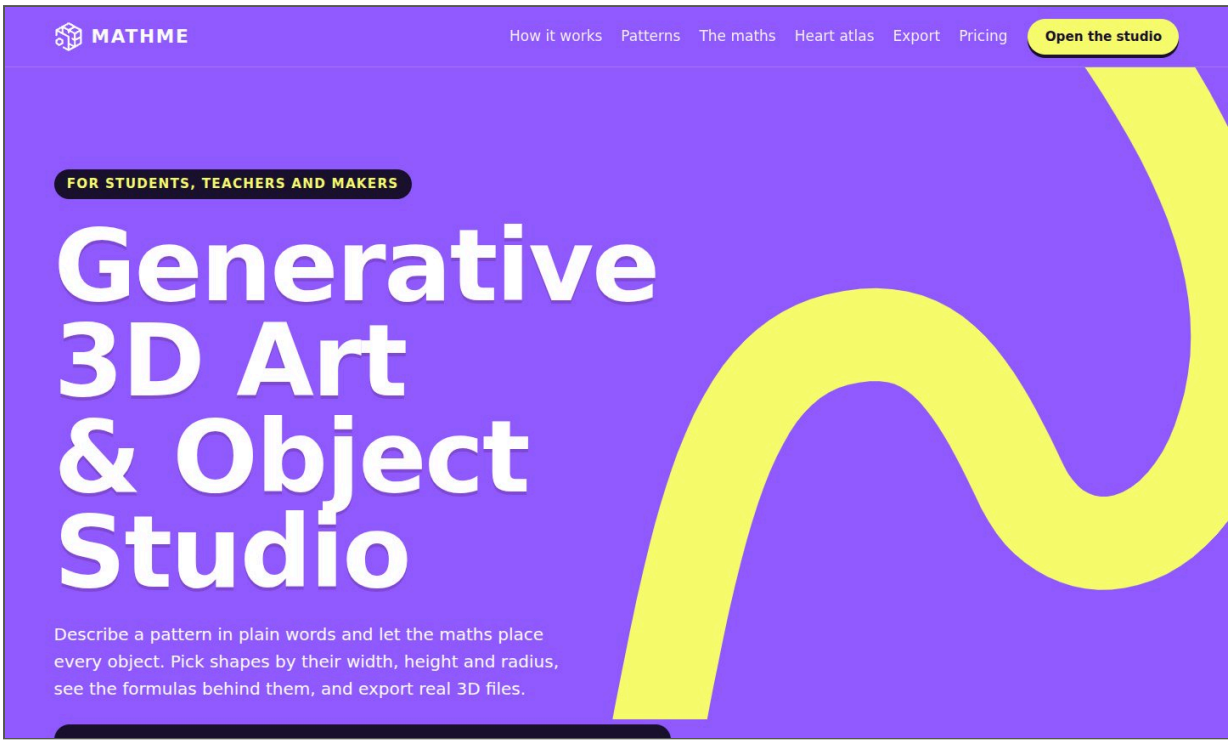


Figure 2.1: Landing Page

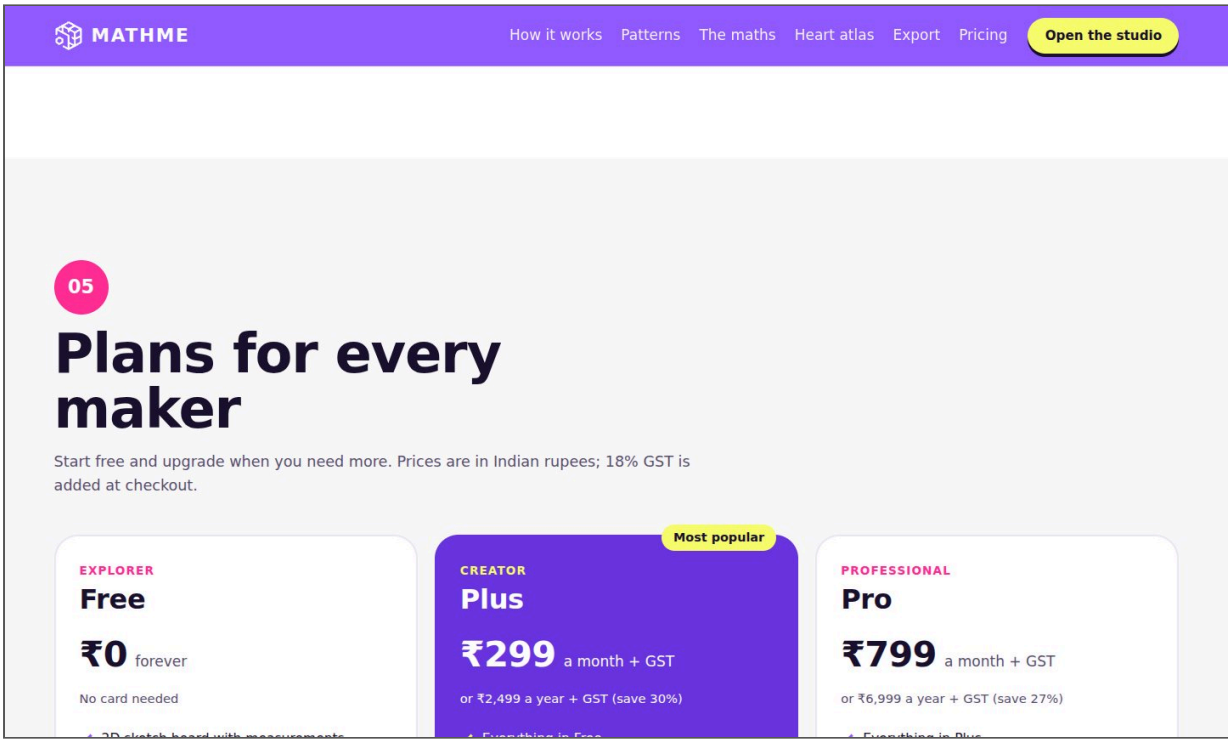


Figure 2.2: Pricing Section of the Landing Page

CHAPTER III: EXPERIENTIAL LEARNING

This chapter describes the work carried out during the internship and the learning gained from it. It begins with the weekly work log, then follows the stages of the software development life cycle: requirement analysis, system design, implementation of each module, the output screens and testing. It ends with the deployment plan, the security measures taken and a summary of what was learnt.

3.1 Weekly Work Log

The internship was planned as ten weeks of part-time work, about nine hours a week, for a total of 90 hours. Table 3.1 lists the work done in each week. The dates and the hours actually spent are filled in by hand and signed off by the guide.

Table 3.1: Weekly work log

Week	Dates	Work done	Hours
1		Studied the project brief; wrote the plan; set up the repository, React + TypeScript + Vite, Python service, linting, tests and continuous integration	
2		Built the maths engine: shapes described by width, height and radius; the seven patterns; variation and symmetry; formulas for the Learn panel	
3		Built the 3D view with instanced rendering, selection and the move/turn/stretch gizmo; the editor panels; undo and redo	
4		Added custom shapes (spun and extruded outlines, 3D text, graph surfaces), the plain-language recipe box, the ideas gallery, autosave and project files	
5		Implemented exports: GLB, STL, OBJ, PNG and the PDF project sheet; started the Python geometry service (measure, join, cut)	
6		Finished the geometry service and print-ready exports; built the human-heart slice atlas and the any-angle cut tool	

Week	Dates	Work done	Hours
7		Redesigned the interface in the brand colours; sidebar, settings page and logo; home page with project list and chat box; landing page	
8		Added the pencil tool and the 2D sketch board (points, lines, circles, measurements, rules, SVG/PNG/PDF/DXF export, 2D-to-3D)	
9		Built the account service: Google sign-in, cloud saves, plans and limits, Cashfree checkout, GST invoices, Campus licences and the owner dashboard	
10		Security review and fixes; browser tests for every flow; deployment guide, policy drafts, landing pricing; this report	
		Total	**90**

3.2 Requirement Analysis

The requirements were collected from the project brief, from the needs of the target users described in Chapter II, and from the business plan. They are divided into functional requirements (what the system must do) and non-functional requirements (how well it must do it).

3.2.1 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement
FR1	The user can create, open, rename, duplicate and delete projects; projects save automatically.
FR2	The user can add shapes described by plain measurements (width, height, depth, radius).
FR3	The user can copy a shape into a spiral, grid, circle, wave, radial rings, sphere or random-scatter pattern.

ID	Requirement
FR4	The system shows the formula of each pattern and a worked example for any chosen object.
FR5	The system works out the volume and surface area of shapes and of the whole model, with working.
FR6	The user can type a recipe such as “100 spheres → spiral → radius 20” to build a scene.
FR7	The user can draw shapes with a pencil and sketch in 2D with lines, circles and rules.
FR8	The user can export GLB, STL, OBJ, PNG, PDF, SVG and DXF files and a project file.
FR9	The user can open a real human-heart model and cut it along standard or custom planes.
FR10	The user can sign in with Google and save projects in the cloud.
FR11	The system enforces the limits of each plan and explains which plan unlocks a feature.
FR12	The user can buy a plan; the system adds GST, takes payment through Cashfree and issues an invoice.
FR13	Teachers and students can join a class with a code; students can share projects with their class.
FR14	The owner can see sign-ups, revenue and churn, answer enquiries and manage school licences.

3.2.2 Non-functional Requirements

Table 3.3: Non-functional requirements

Quality	Requirement
Usability	Plain words instead of technical terms; every control has a short help text; works for a 10th-grade student
Performance	Smooth 3D view with thousands of objects (instanced rendering); first screen loads quickly

Quality	Requirement
Portability	Runs in any modern browser on Windows, macOS, Linux, Chromebooks and tablets; no installation
Reliability	Autosave, undo/redo, version checks on cloud saves; payments processed exactly once
Security	Secure sign-in, signed payment webhooks, protection against cross-site requests
Maintainability	Typed code, small modules, one source of truth for prices, automated tests
Compliance	GST invoices numbered per financial year; refund, privacy and terms pages
Accessibility	Keyboard support, labelled controls, readable colours, layouts that fit a phone screen

3.2.3 Hardware and Software Requirements

Table 3.4: Hardware and software requirements

Item	For using MathMe	For developing MathMe
Processor	Any dual-core processor	Quad-core processor
Memory	4 GB RAM	8 GB RAM or more
Graphics	WebGL-capable graphics	WebGL-capable graphics
Operating system	Any (Windows, macOS, Linux, ChromeOS)	Windows, macOS or Linux
Software	Chrome, Edge, Firefox or Safari	Node.js 22, Python 3.11+, Git, VS Code
Network	Internet for the first load; cloud features need a connection	Internet for packages and deployment

3.3 System Design

3.3.1 System Architecture

MathMe follows a **three-tier, service-based architecture** (Figure 3.1). The **presentation tier** is the web application running in the browser. It contains the whole maths engine, so the studio works even without a server, and it keeps device projects in browser storage.

The **application tier** has two services, each with a narrow job:

- the **account service**, a FastAPI application designed to run as a Cloudflare Python Worker, which handles sign-in, cloud projects, plans, payments and school licences;
- the **geometry service**, a FastAPI application in a Docker container, which performs heavy and exact geometry work with trimesh and manifold3d.

The **data tier** is Cloudflare D1 (an SQLite database) for records and R2 for project files. External services are Google (sign-in), Cashfree (payments) and Resend (email).

The browser talks only to the account service, under the same web address (/api/*). The account service checks the user's plan and daily quota before passing a geometry job to the geometry service with a shared secret, so the quotas cannot be bypassed.

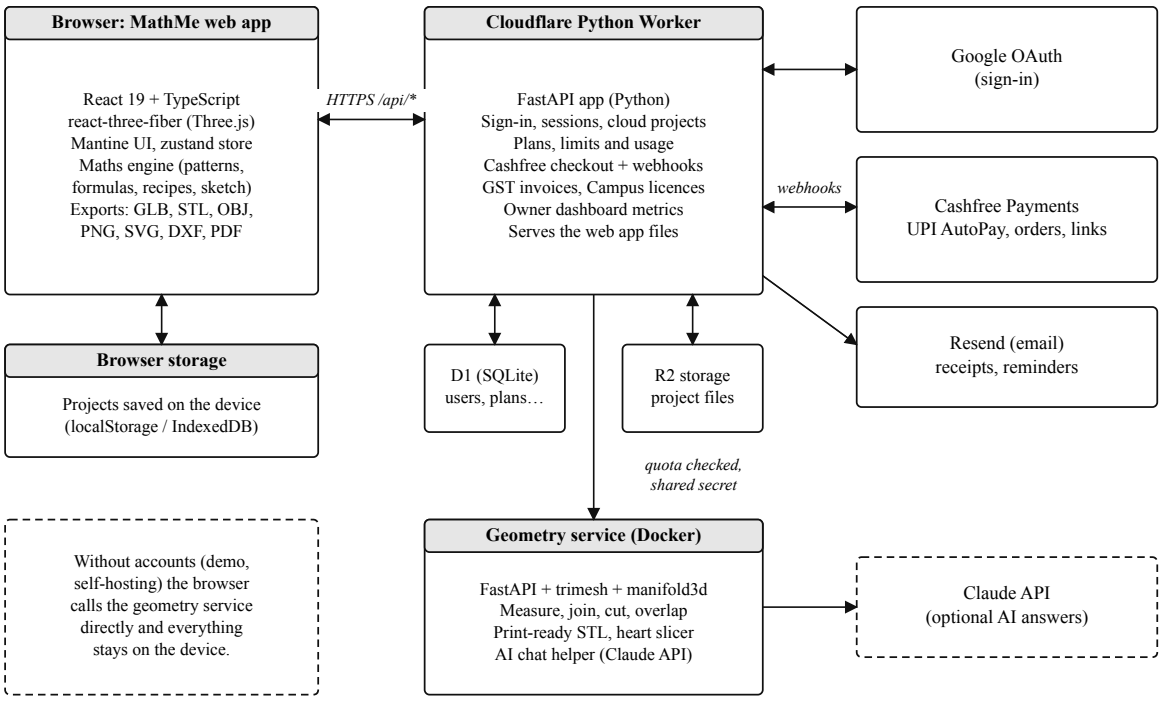


Figure 3.1: System architecture of MathMe

3.3.2 Data Flow Diagrams

The **context diagram (Level 0 DFD)** in Figure 3.2 shows MathMe as a single process and its five external entities: the student or maker, the teacher, the owner, Google, and Cashfree/Resend. Students send shapes, patterns, recipes and sketches, and receive the 3D view, formulas and exported files. Teachers send join codes and receive the class page. The owner manages licences and receives metrics. Google confirms identity, and Cashfree reports payment events.

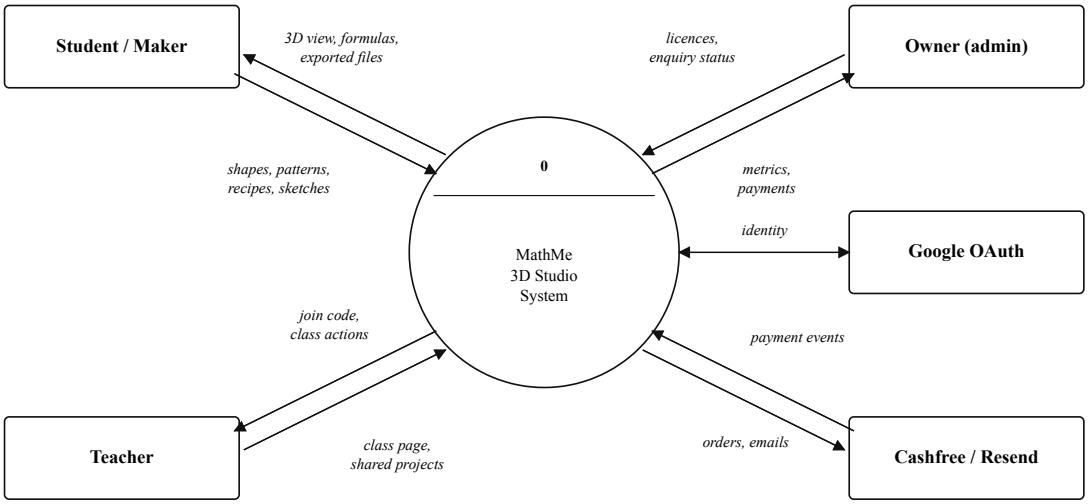


Figure 3.2: Context diagram (Level 0 DFD)

The **Level 1 DFD** in Figure 3.3 breaks the system into seven processes:

- 1.0 build the scene;
- 2.0 export files;
- 3.0 run geometry jobs;
- 4.0 sign in and save to the cloud;
- 5.0 billing and GST;
- 6.0 the Campus class;
- 7.0 the owner dashboard.

It also shows four data stores:

- D1: device projects in the browser;

- D2: users and cloud projects;
- D3: payments and invoices;
- D4: organisations and their members.

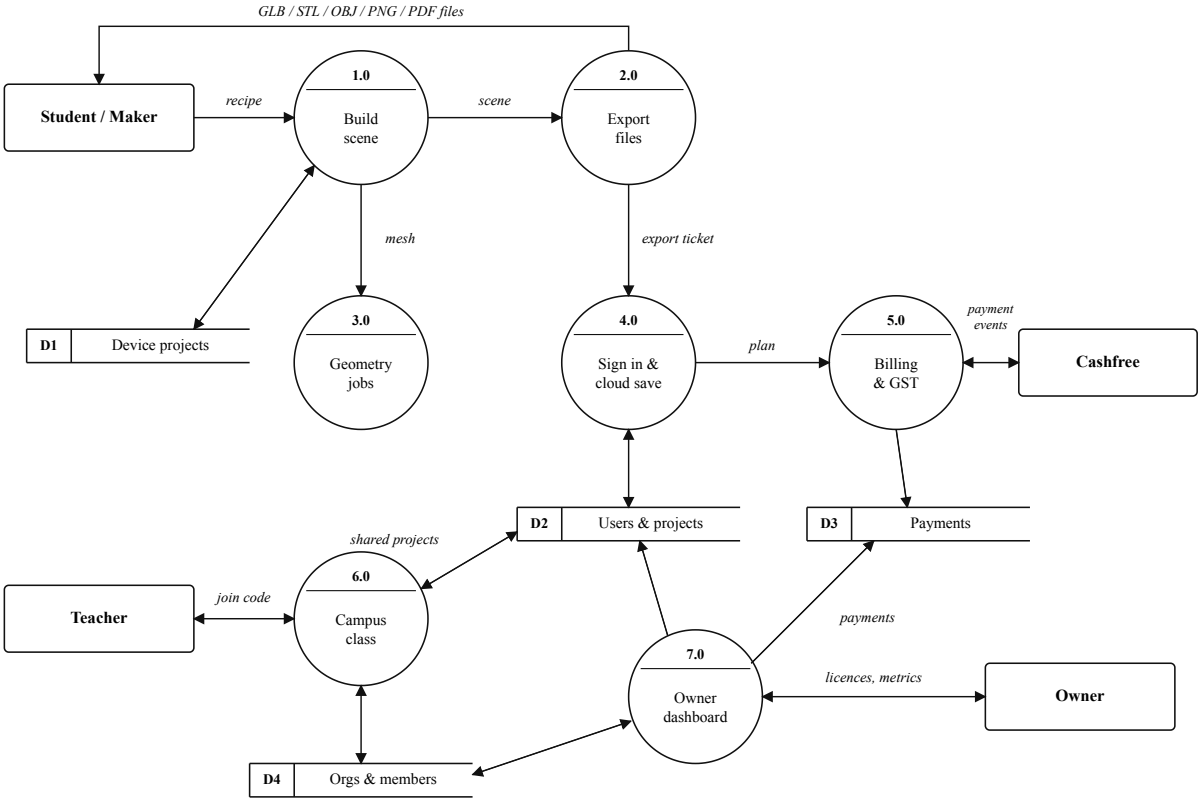


Figure 3.3: Level 1 data flow diagram

3.3.3 Database Design (E-R Diagram)

The account service stores its data in a relational database (SQLite, run as Cloudflare D1 in production). The same SQL migration file creates the tables in both places. Figure 3.4 shows the entity-relationship diagram of the main tables:

- A **user** has many sessions, projects, subscriptions, payments and usage counters.
- A **payment** has exactly one GST **invoice**.
- An **organisation** (a Campus licence) has many members, through the ORG_MEMBERS table, and can make payments.
- A **project** can be shared with one organisation.

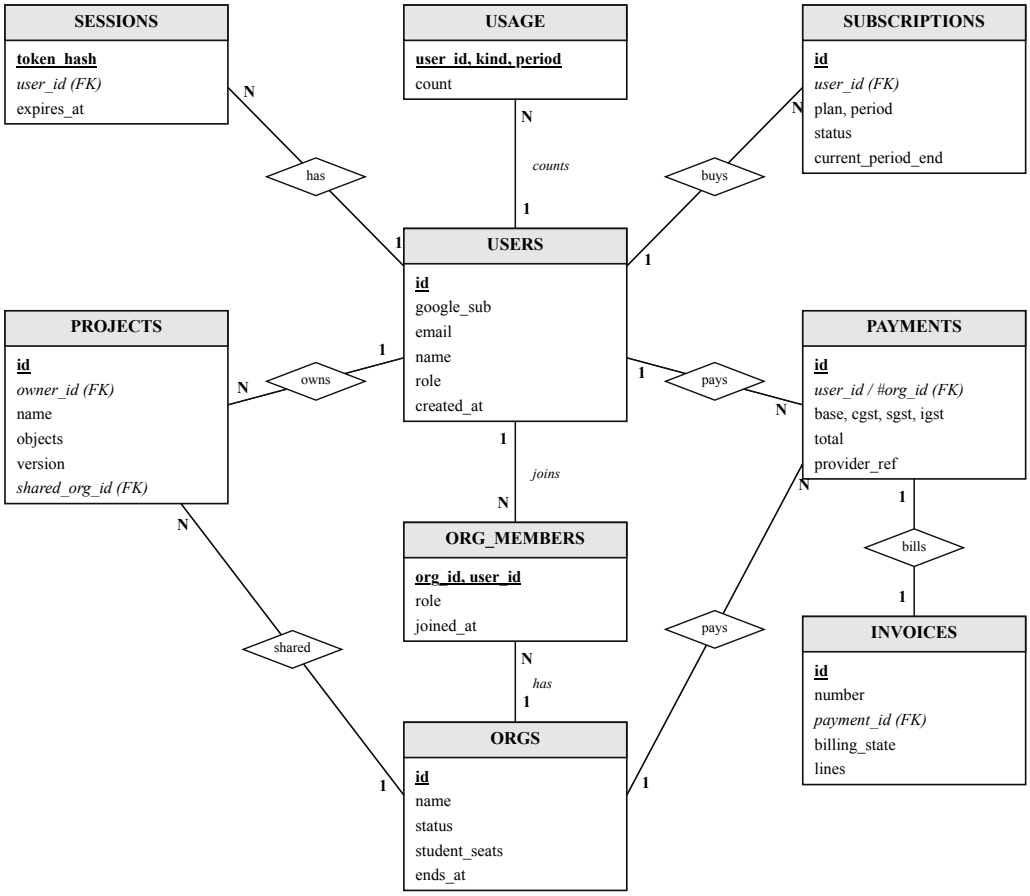


Figure 3.4: Entity-relationship diagram of the account database

3.3.4 Use Case Diagram

Figure 3.5 shows the three kinds of users and what each one can do. The **student** creates and explores; the **teacher** can do everything a student can and also manages a class; the **owner** manages licences, enquiries and the business numbers.

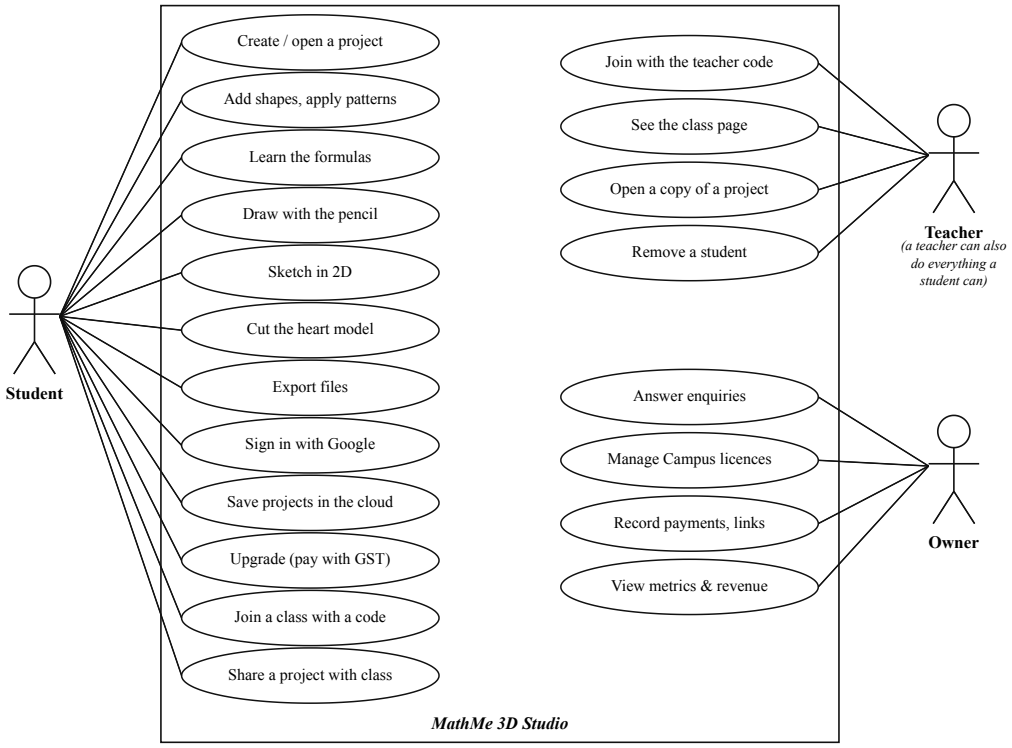


Figure 3.5: Use case diagram

3.3.5 Module Structure

The source code is organised as a monorepo (Figure 3.6). The maths engine (web/src/engine) has no user-interface code, so it can be tested quickly and reused. The application code is split into the 3D viewport, the user interface, the state stores and the exporters. The two Python services each have their own routes, logic and tests. One shared file (shared/plans.json) holds every price and limit.

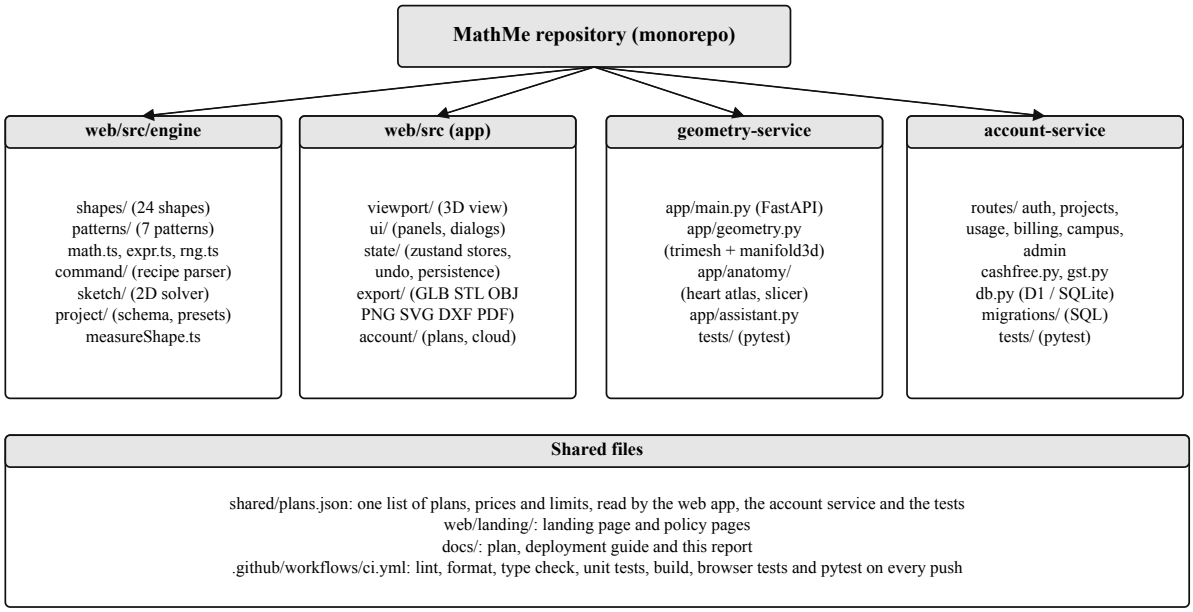


Figure 3.6: Module structure of the MathMe repository

3.4 Implementation

This section describes how each module was implemented, with short extracts from the source code. The screenshots of the working application are collected two to a page in Section 3.5 (Output Screens).

3.4.1 Home Page and Project Management

The home page (Figure 3.8) is the first screen after the landing page. A chat box asks “What shall we make today?”: the user can describe an artwork in words, start drawing with the pencil, or choose a recipe. Below it, the user’s projects appear as cards with a small picture of each scene, and a gallery offers ready-made ideas such as a sunflower, a DNA double helix, an atom, a snowflake and the human heart.

Each project is stored as a JSON document that follows a versioned schema, checked with the Zod validation library when it is opened. Projects save automatically a moment after every change, and the picture on each card is captured from the 3D view.

3.4.2 Maths Engine and Patterns

The maths engine is the heart of MathMe. A **pattern node** combines a source shape, a pattern type, a count and the pattern’s parameters. When any value changes, the engine calls the pattern’s generate function, which returns a position and an orientation for every copy.

Variation rules then change the size, rotation and colour of each copy (for example “scale 0.5–2” or a rainbow of colours), and the symmetry modifier can mirror or rotate the whole result.

Every pattern is a small, self-contained definition with its own fields, help texts, default values and an explain function that produces the formulas for the Learn panel. Table 3.5 lists the seven patterns and their core formulas.

Table 3.5: Patterns and their formulas

Pattern	Rule for object number i
Spiral	$\theta = \text{start} + i \times \text{step}$; $r = R \times i \div (n - 1)$; $x = r \cos\theta$, $z = r \sin\theta$, $y = i \times \text{rise}$
Sunflower spiral	$\theta = i \times 137.5°$ (golden angle); $r = R \times \sqrt{(i \div (n - 1))}$
Grid	column = i mod columns, row = $\lfloor i \div \text{columns} \rfloor$; x = column × gap, z = row × gap (centred)
Circle	$\theta = i \times 360° \div n$; $x = R \cos\theta$, $z = R \sin\theta$
Wave	$y = A \times \sin(k \times \text{distance from the centre})$ on a grid of points
Radial rings	ring k has (first ring) × k objects at radius $r = k \times \text{gap}$; $\theta = 360° \times j \div$ (objects in ring)
Sphere (Fibonacci)	$h = 1 - 2(i + 0.5) \div n$; ring = $\sqrt{(1 - h^2)}$ (Pythagoras); $\theta = i \times 137.5°$
Random scatter	positions from a seeded random generator, so the same seed gives the same result

The extract below is the generate function of the spiral pattern. For each object it works out the angle θ and the distance r, and then uses $x = r \cos\theta$ and $z = r \sin\theta$, exactly the formulas a student learns in trigonometry.

Extract from web/src/engine/patterns/spiral.ts

```
generate(p, n) {
  const out: BasePlacement[] = [];
  for (let i = 0; i < n; i++) {
    const { r, thetaDeg, y } = spiralPoint(p, i, n);
    const th = thetaDeg * DEG;              // degrees → radians
    out.push({ position: [r * Math.cos(th), y, r * Math.sin(th)],
               align: yaw(-th) });          // face along the spiral
  }
  return out;
}
```

Figure 3.9 shows the example from the project brief, "100 spheres → spiral → radius 20 → rotation 30 → scale 0.5–2", typed into the recipe box. Figure 3.10 shows the Pattern panel, where every value can be changed with a slider or by typing a number.

To keep the view smooth with thousands of objects, all copies of the same shape are drawn with **instanced rendering**: the graphics card receives one shape and a list of positions, instead of thousands of separate objects. A scene of 20,000 objects still turns smoothly on an ordinary laptop.

3.4.3 Learn and Measure Panels

The **Learn** panel (Figure 3.11) explains the idea behind the selected pattern in one or two sentences, lists its formulas, and works them out for any object the user chooses. For example, it shows $\theta = 10 \times 30° = 300°$ and $x = 2.02 \times \cos 300° = 1.01$ for object number 10. The aim is that students see the same formulas from their textbook producing the artwork in front of them.

The **Measure** panel (Figure 3.12) gives the volume and surface area of one object with full working (for a sphere, $V = 4/3 \times \pi \times r^3$). It then explains how scaling by k changes the area by k^2 and the volume by k^3, so it can add up the whole pattern. For the exact volume of overlapping objects, the panel calls the geometry service, which joins all the shapes into one solid first.

3.4.4 Recipes, Ideas and the Assistant

The **recipe box** at the top of the studio accepts one-line descriptions such as “300 cones → sphere → radius 10” or “12 stars in a circle, radius 8”. A small parser written for this project splits the text into tokens, recognises numbers, shapes, patterns and settings (including synonyms such as “rotation” for the turn between objects), and reports a clear message if something is not understood.

The **ideas gallery** (Figure 3.13) opens ready-made scenes such as a sunflower, a DNA helix or a wave field, which students can study and change. On the home page, the chat box sends the user’s request to an optional AI helper through the geometry service; when the AI is not available, a built-in local matcher still turns common requests into recipes.

3.4.5 Custom Shapes

Besides the 21 built-in shape types (cuboid, sphere, cylinder, cone, torus, prism, pyramid, the five Platonic solids, torus knot, capsule and more), MathMe lets the user make their own shapes. A **spun shape** is drawn as half an outline and spun around an axis, like clay on a potter’s wheel, to make vases, bowls and goblets. An **extruded shape** pushes a flat outline up into a solid. There are also 3D text, graph surfaces of the form y = f(x, z) and parametric surfaces. Groups of shapes can be saved under “Mine” and used as the source of a pattern.

3.4.6 Exports

All exports are made inside the browser, so they work without a server (Figure 3.14). Table 3.6 lists the formats. Each exporter converts the instanced scene into ordinary meshes, applies the right units (metres for GLB, millimetres for STL) and names every part so that it can be found again in other programs.

Table 3.6: Export formats

Format	Used for	Notes
GLB (glTF 2.0)	Blender, Unity, Unreal, web pages	One mesh per object, in metres
STL	3D printers (slicer software)	In millimetres; print-ready joined solid via the geometry service
OBJ	Almost every 3D program	Simple text format

Format	Used for	Notes
PNG	Posters and slides	HD, 4K or square; optional see-through background
PDF	Class reports	Project sheet with views, settings and the maths
SVG, DXF, PDF	2D sketches	Vector drawings for CAD and laser cutting
.mathme.json	Saving and sharing projects	Opens again in MathMe with File → Open

3.4.7 Geometry Service (Python)

Some tasks need exact geometry that is too heavy or too complex for the browser. These run in the **geometry service**, a FastAPI application with the following endpoints:

- **/analyze:** volume, surface area and bounds of the whole model.
- **/boolean:** join, cut or overlap two shapes.
- **/export:** print-ready files.
- **/slice:** cut the heart model.
- **/assist:** the AI helper.

The browser sends the scene as a GLB file, and the service loads it with the trimesh library and uses the manifold3d engine for robust boolean operations. Inputs are limited in size, triangle count and running time, so that one request cannot overload the server.

3.4.8 Human Heart Slice Atlas

To show how the same tools serve science teaching, MathMe includes a real, scan-based human heart from the **Human Reference Atlas** of the HuBMAP consortium (licensed CC BY 4.0), with 51 named structures. The cut tool (Figure 3.15) slices the model along 21 standard planes used in anatomy and cardiac imaging, such as the four-chamber, three-chamber, two-chamber and short-axis views, or along any angle chosen with the sliders. The plane is shown as an equation $n \cdot x = d$, again linking the anatomy to the mathematics of planes. The Python side also produces a labelled PDF slice atlas of about 50 pages.

3.4.9 Pencil Drawing

The pencil (Figure 3.16) lets beginners draw directly in the 3D view. A closed loop drawn on the floor becomes a solid that can be pushed up; an open line becomes a 3D pen “tube”; and a “lines” mode lets the user click corner points to make straight-edged shapes. The drawn points are simplified and smoothed before the solid is built, so freehand drawings still give clean shapes.

3.4.10 2D Sketch Board

The 2D sketch board (Figure 3.17) is a geometry board in the style of a school notebook. The user places **node points**, joins them with **lines**, and draws **circles by radius**. Points snap to the grid, to other points and to the middle of lines. Selecting a closed shape shows its area and perimeter; selecting a line shows its length and angle; selecting two lines shows the angle between them.

The user can add geometric **rules**: horizontal, vertical, fixed length, parallel, perpendicular, equal and fixed radius. A small **constraint solver**, written for this project, nudges the points again and again, like a network of springs, until every rule holds, and it reports any rules that conflict. Sketches can be exported as SVG, PNG, PDF and DXF, or turned into 3D solids by pushing them up (extrusion) or spinning them (lathe).

3.4.11 User Interface and Settings

The interface was redesigned during the internship to be minimal and friendly:

- a collapsible sidebar in the style of modern chat applications;
- a details panel that can be hidden;
- consistent brand colours taken from the MathMe presentation;
- a welcome tour and a help window with a glossary of maths words.

The Settings page groups the project options (name, author, units and background), appearance (light or dark theme), 3D-view options, data management and an About section.

The Help window explains every tool in short sentences, lists the keyboard shortcuts and includes a glossary of “maths words” (such as radius, angle, sine, volume and symmetry) in simple language, so that students can look up a term without leaving the studio.

3.4.12 Accounts, Plans and Limits

Accounts are switched on by a build setting (VITE_ACCOUNTS), so the free demo and self-hosted copies keep working without a server. When accounts are on, users sign in with **Google** using the secure authorisation-code flow: the browser never sees Google's tokens. The server creates a session stored in an HttpOnly cookie, and only a hash of the session token is kept in the database.

Each plan's limits come from the shared plans file. Before an export, a geometry job or an AI message, the app asks the server for a "ticket", and the server counts the use with an atomic database update. When a limit is reached, or a feature belongs to a higher plan, the user sees a friendly prompt that names the feature and the plan that unlocks it (Figure 3.18). The plans window (Figure 3.19) compares Free, Plus and Pro with monthly and yearly prices.

3.4.13 Checkout, GST and Invoices

Checkout asks for the details needed for a GST invoice: name, mobile number, state and an optional GSTIN for businesses (Figure 3.20). The buyer's state decides the tax lines: within the seller's state, 9% CGST and 9% SGST; otherwise, 18% IGST. All amounts are handled in **paise** (whole numbers), so there are no rounding errors from decimal fractions. Table 3.7 shows the prices with GST.

Table 3.7: Prices with 18% GST

Plan	Price before GST	GST (18%)	Total charged
Plus, monthly	₹299.00	₹53.82	₹352.82
Plus, yearly	₹2,499.00	₹449.82	₹2,948.82
Pro, monthly	₹799.00	₹143.82	₹942.82
Pro, yearly	₹6,999.00	₹1,259.82	₹8,258.82
Campus, yearly	₹14,999.00	₹2,699.82	₹17,698.82

Extract from account-service/src/account_service/gst.py

```
def add_gst(base: int, seller_state: str, buyer_state: str) -> Tax:
    """Base price (paise, before GST) → the tax lines and the total."""
    tax = (base * RATE_PERCENT + 50) // 100   # 18%, rounded half up to the paisa
    if seller_state.strip().lower() == buyer_state.strip().lower():
        cgst = tax // 2
        return Tax(base, cgst, tax - cgst, 0, base + tax)   # CGST + SGST
    return Tax(base, 0, 0, tax, base + tax)                 # IGST
```

Payments use **Cashfree Payments**. Monthly plans are set up as a subscription mandate (UPI AutoPay, card or e-mandate), and yearly plans as a one-time order. Cashfree informs MathMe of every payment through a **webhook**. The flow is shown in Figure 3.7:

- The server first checks the webhook's signature, a Base64 HMAC-SHA256 of the timestamp and the body, keyed with a secret.
- It rejects messages that are more than five minutes old.
- It records each event id, so that a repeated message is processed only once.
- It then records the payment and issues an invoice numbered per financial year (for example MM/2026-27/000001).
- It emails a receipt. The invoice can be downloaded as a PDF from Settings → Account (Figure 3.21).

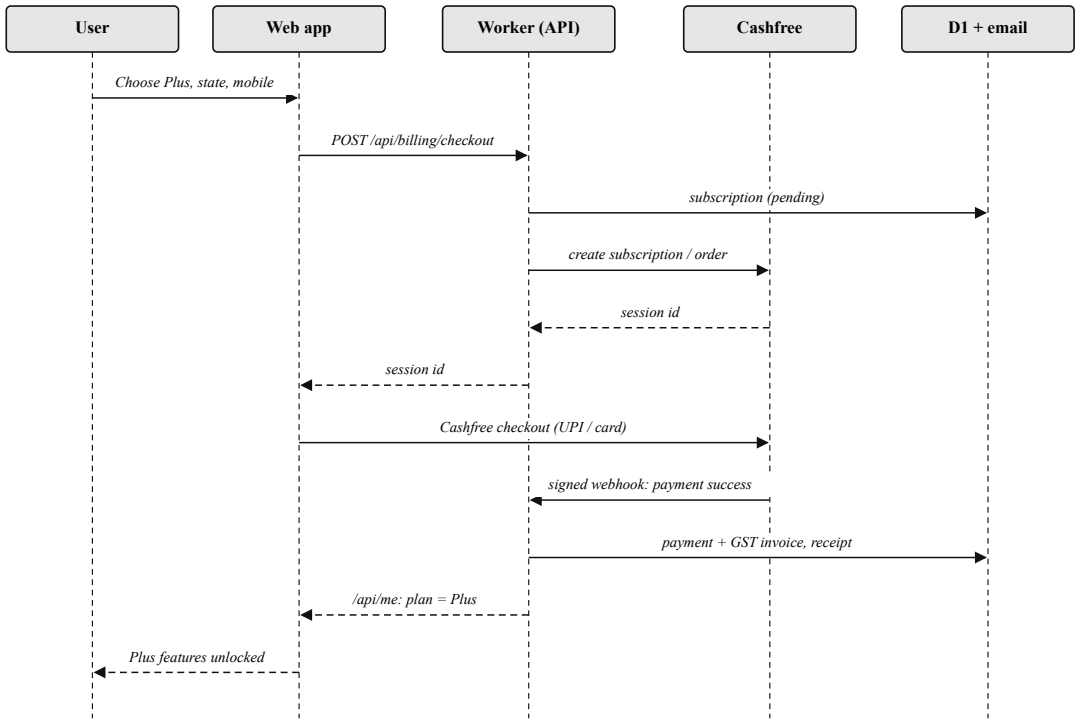


Figure 3.7: Sequence diagram of a Plus checkout

3.4.14 Campus Licences and the Class Page

A school or college starts with an enquiry form on the landing page. The owner turns the enquiry into a **Campus licence** for 100 students and 5 teachers, which creates two join codes, one for teachers and one for students. When a member signs in and types a code, the server checks the seat limits and adds them to the class. Teachers then get Pro features and students get Plus features while the licence is valid.

Students can share a cloud project with their class. The teacher's class page (Figure 3.22) shows the licence, the codes, the shared projects (which the teacher can open as a copy) and the class list, from which a student can be removed.

3.4.15 Owner Dashboard

The owner dashboard (Figure 3.23) is visible only to the accounts listed as owners. It shows:

- sign-ups and weekly active users;
- paying users by plan and the free-to-paid conversion rate;
- monthly recurring revenue (MRR) and 30-day churn;
- revenue before GST;

- Campus pilots and paid licences.

Separate tabs list enquiries, Campus licences (Figure 3.24), with actions to extend a licence, mark it paid or send a payment link, and recent payments. A **revenue calculator** (Figure 3.25) reproduces the scenario from the business plan, ₹86,125 a month or ₹10,33,495 a year, and lets the owner try other numbers of Plus, Pro, Campus and Enterprise customers.

3.4.16 Landing Page, Enquiries and Policy Pages

The public landing page (Figures 2.1 and 2.2) explains MathMe in four steps and ends with the pricing section, whose prices are checked by an automated test against the shared plans file. Schools and companies use its enquiry form to ask for a Campus pilot or an Enterprise quotation; the request is stored for the owner dashboard and emailed to the owner. Because an Indian payment gateway requires them, draft Terms of Use, Privacy, Refund and Cancellation, and Contact pages were also written; they are marked to be reviewed by a lawyer before launch.

3.5 Output Screens

The following pages show the output screens of MathMe, printed in colour, two screens to a page. Each screenshot is named after the screen it shows, and Section 3.4 refers to them by figure number.

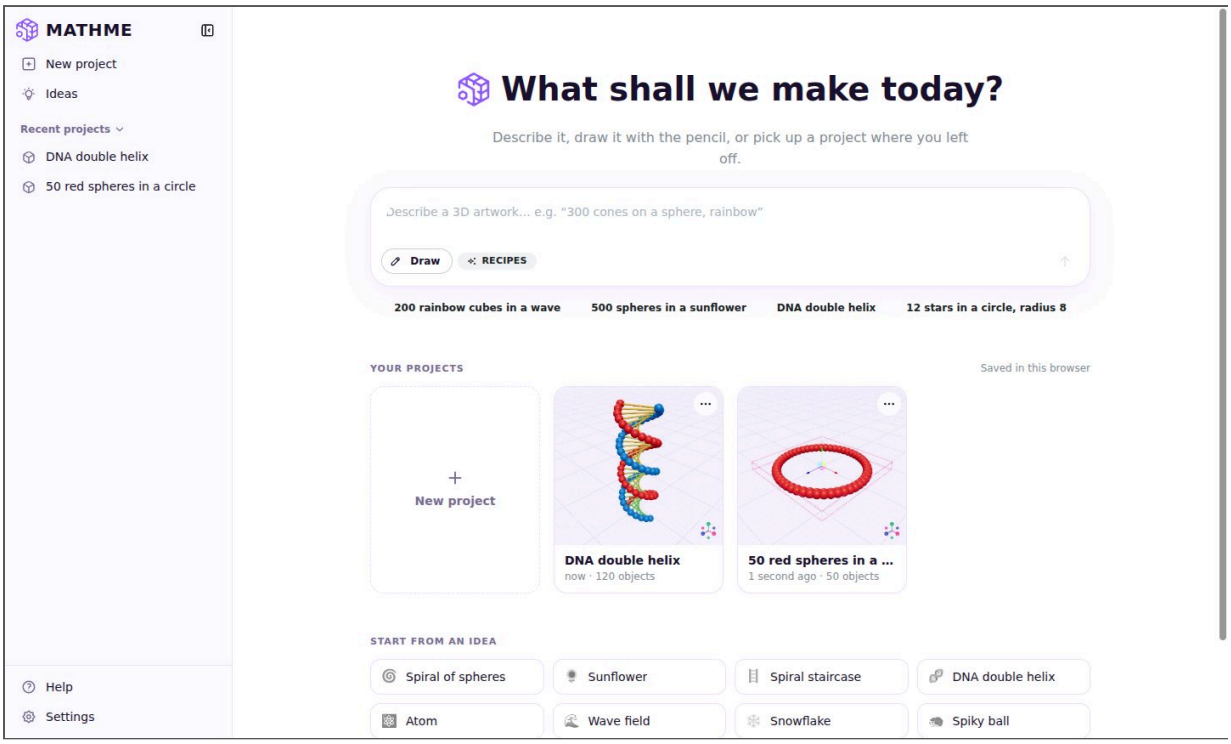


Figure 3.8: Home Page

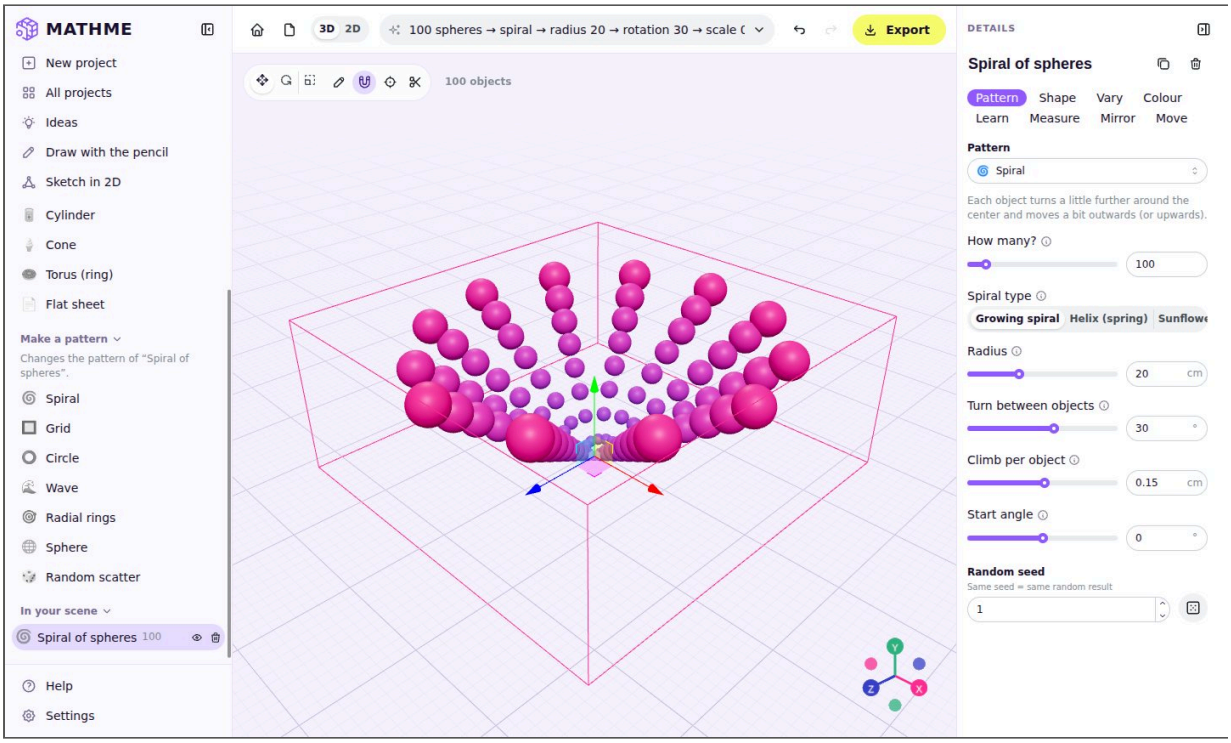


Figure 3.9: 3D Studio Page (100 Spheres in a Spiral)

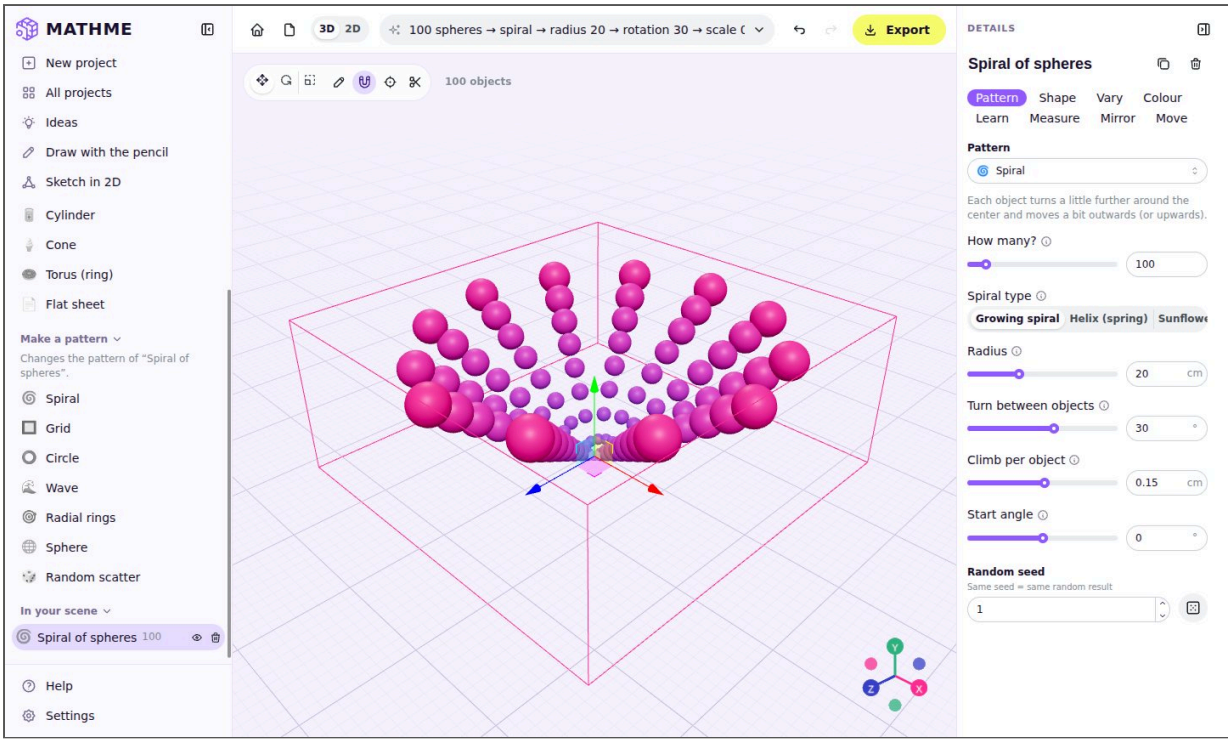


Figure 3.10: Pattern Panel

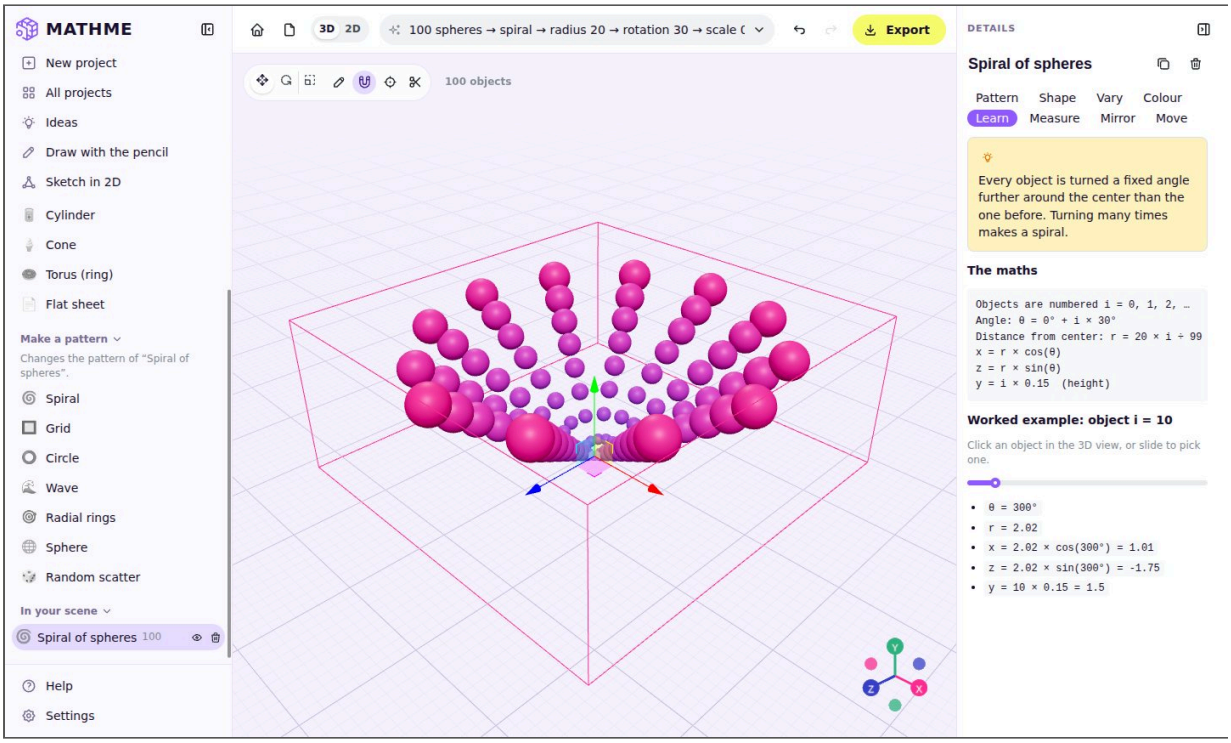


Figure 3.11: Learn Panel

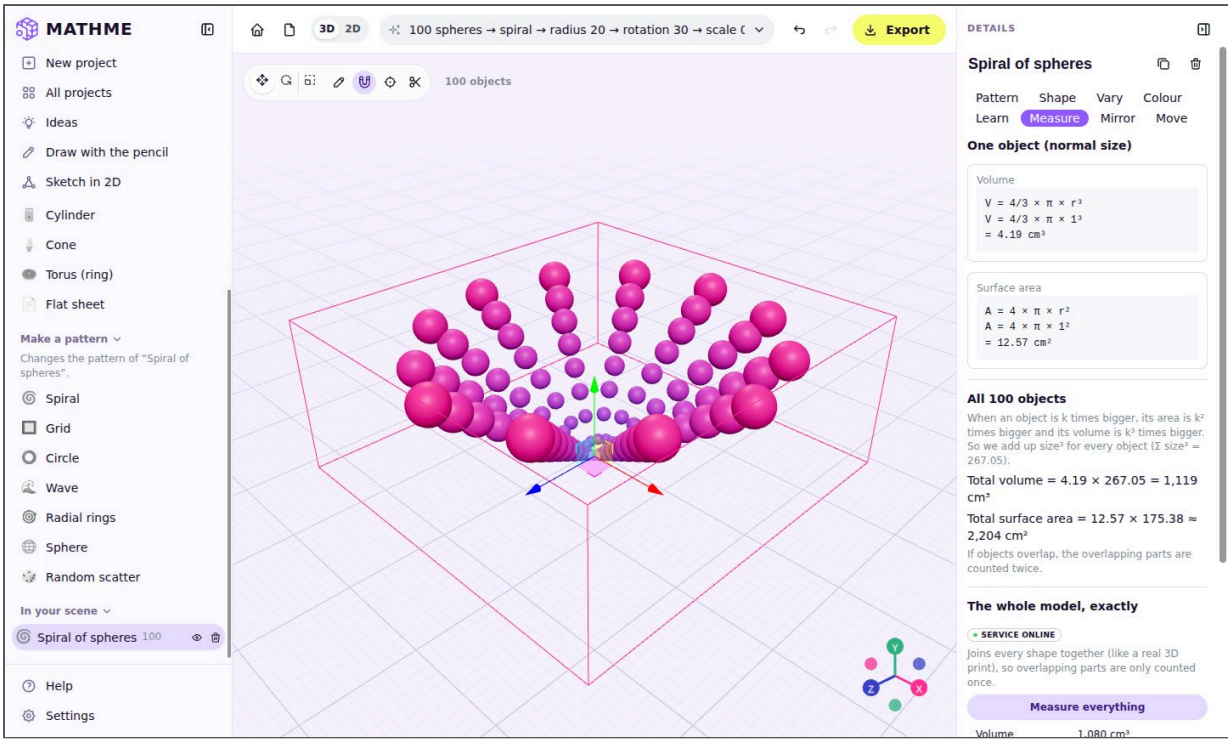


Figure 3.12: Measure Panel

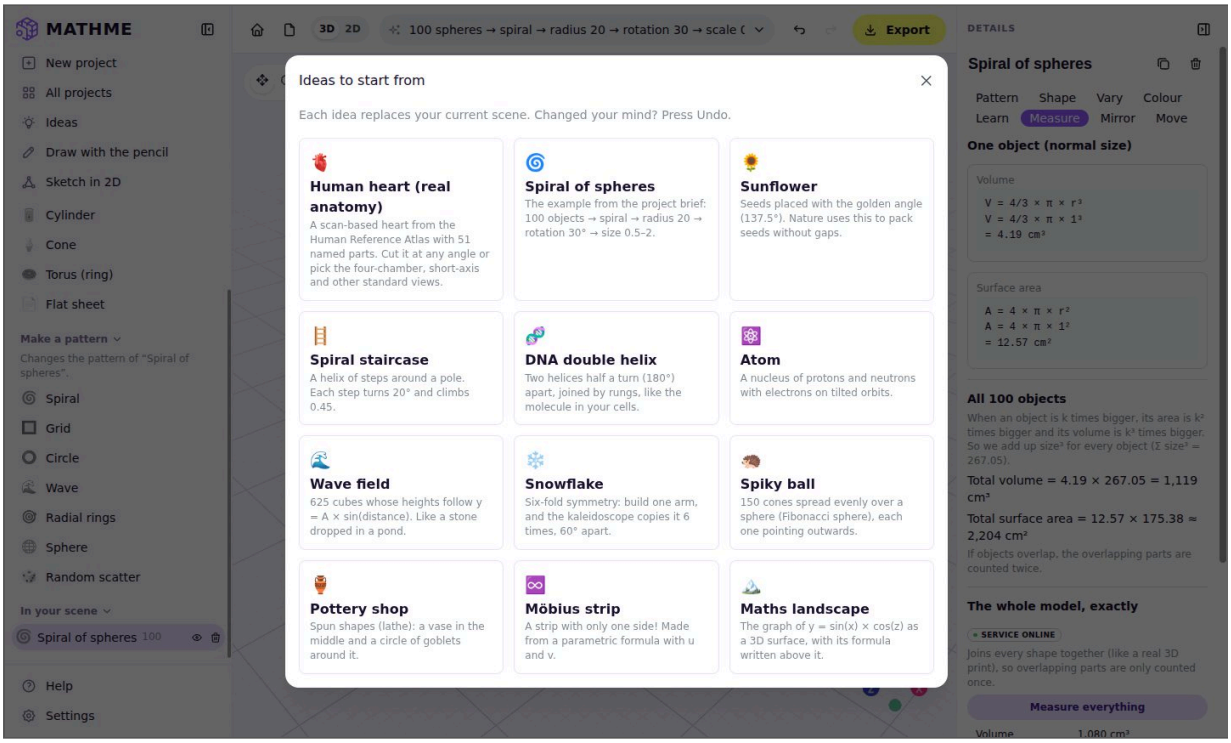


Figure 3.13: Ideas Gallery

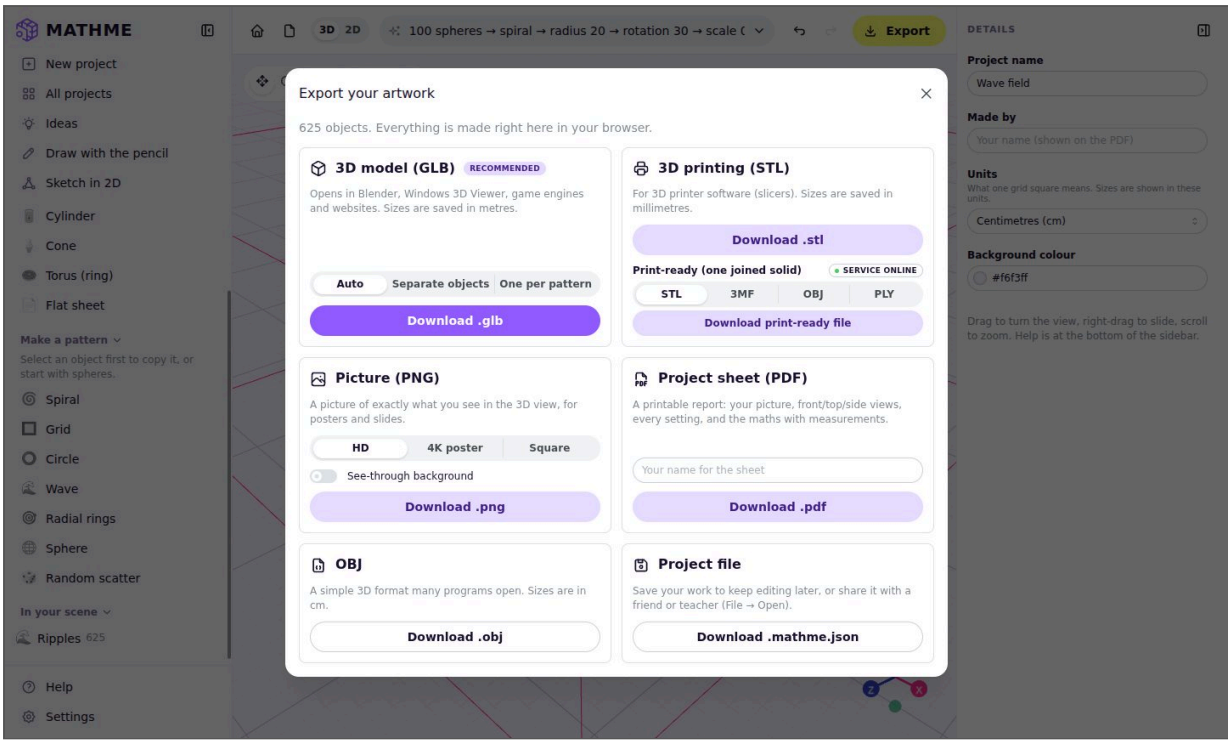


Figure 3.14: Export Page

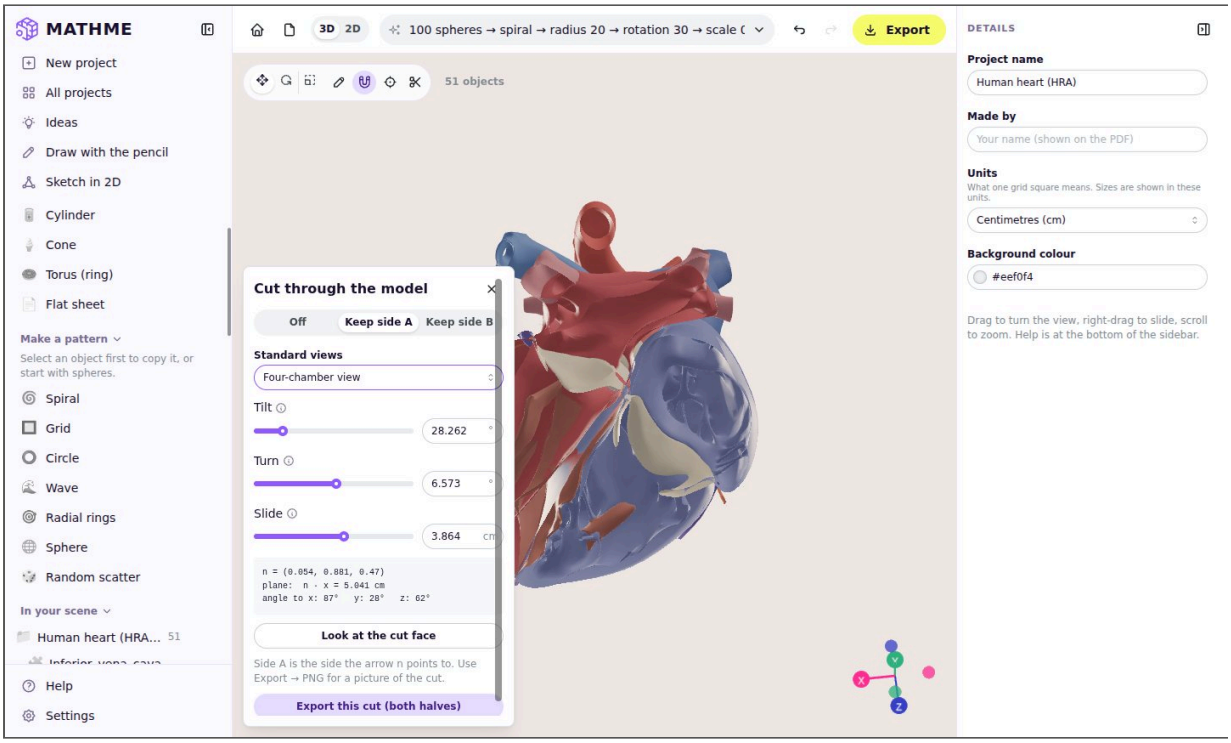


Figure 3.15: Heart Cut Page (Four-Chamber View)

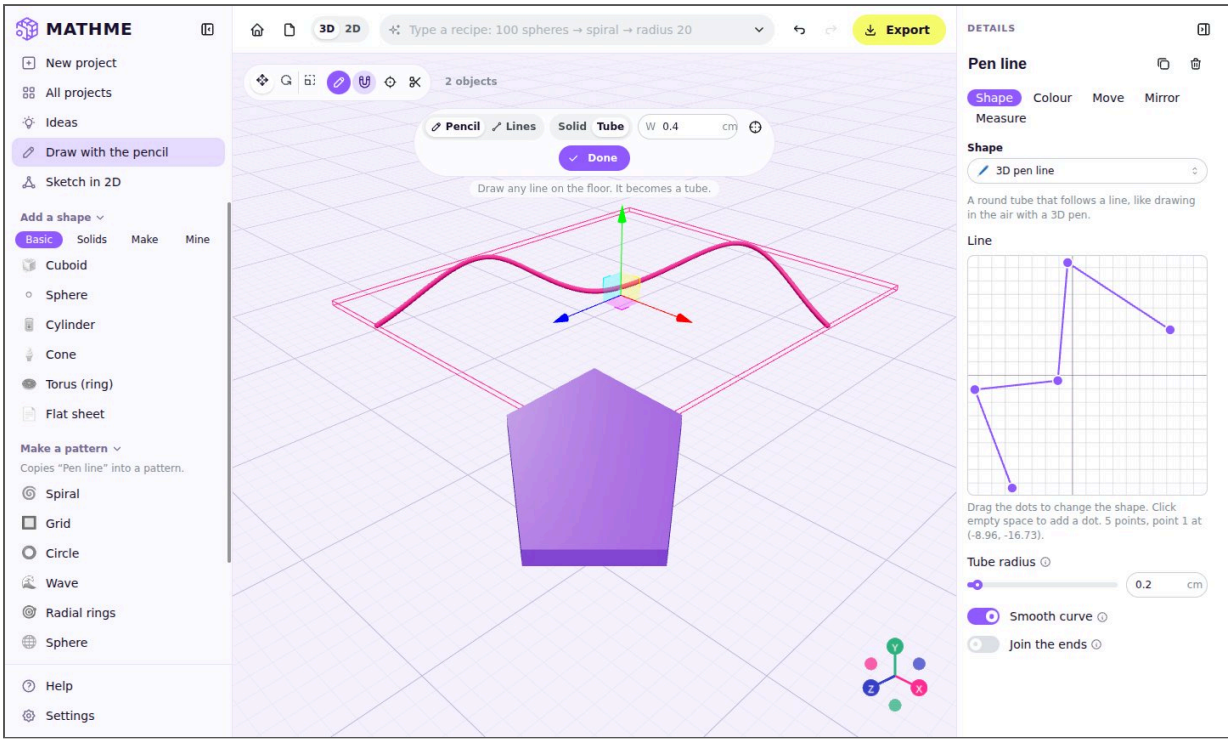


Figure 3.16: Pencil Drawing Page

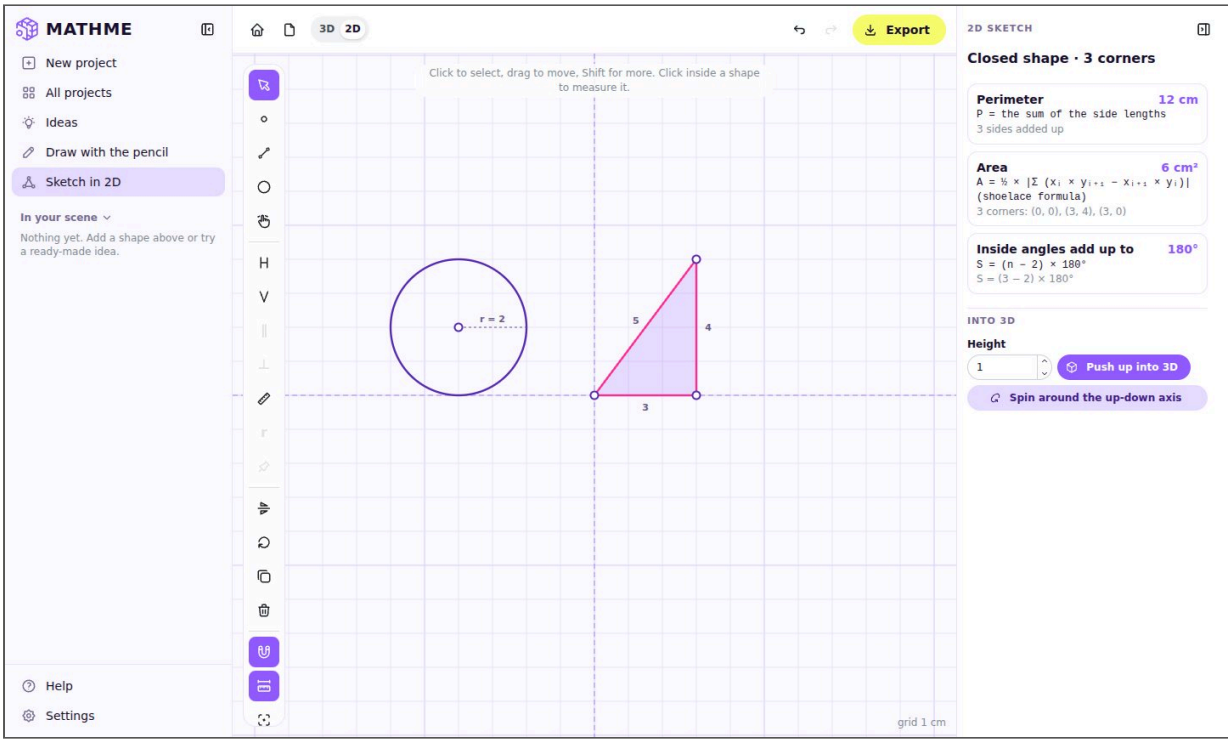


Figure 3.17: 2D Sketch Board

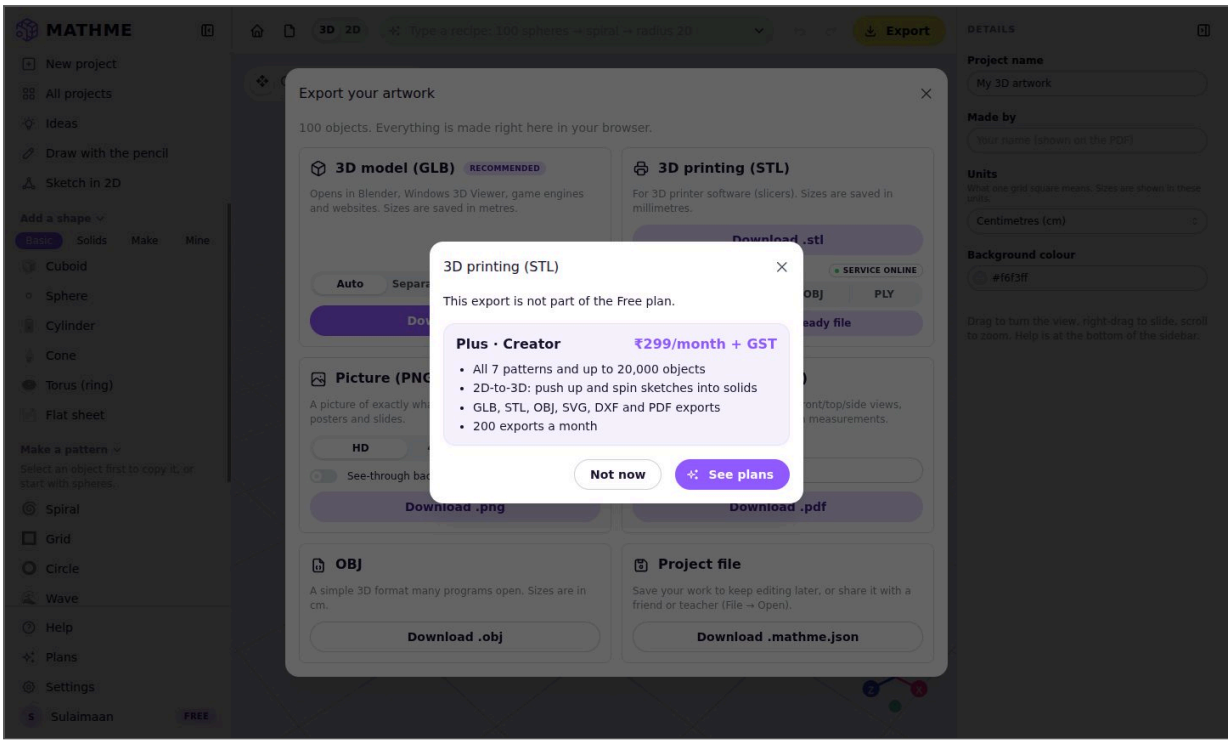


Figure 3.18: Upgrade Prompt

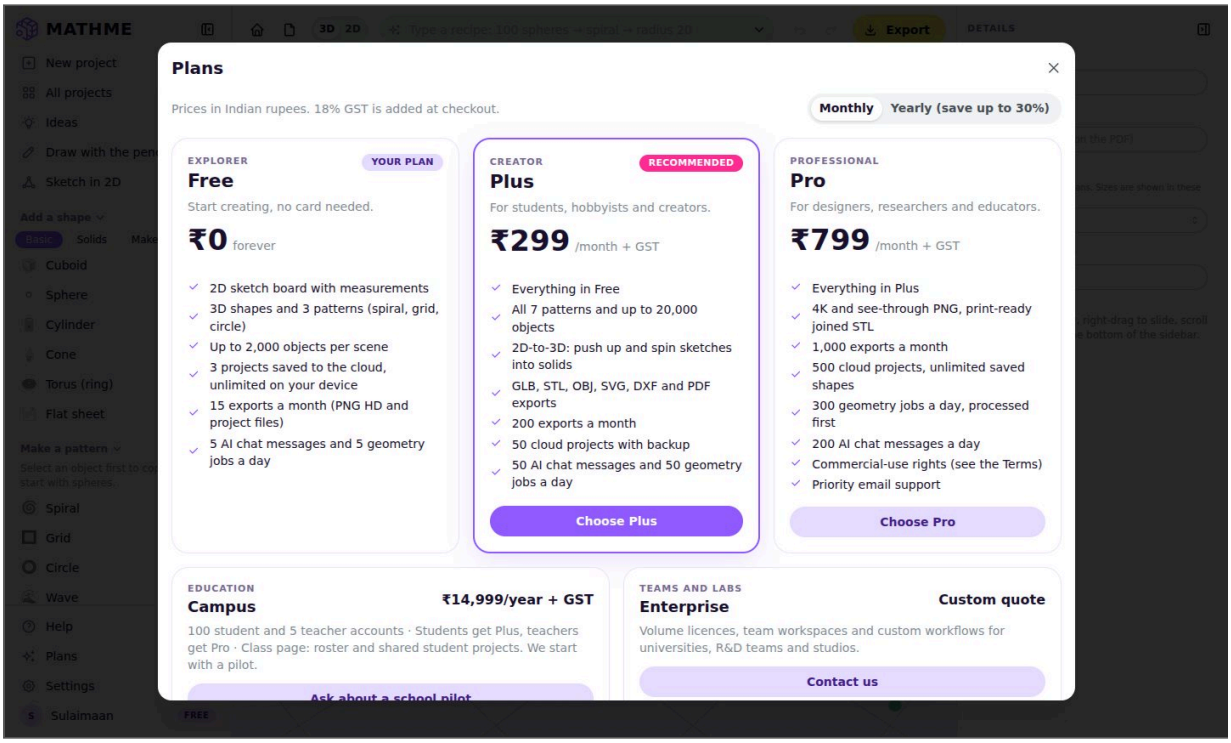


Figure 3.19: Plans Page

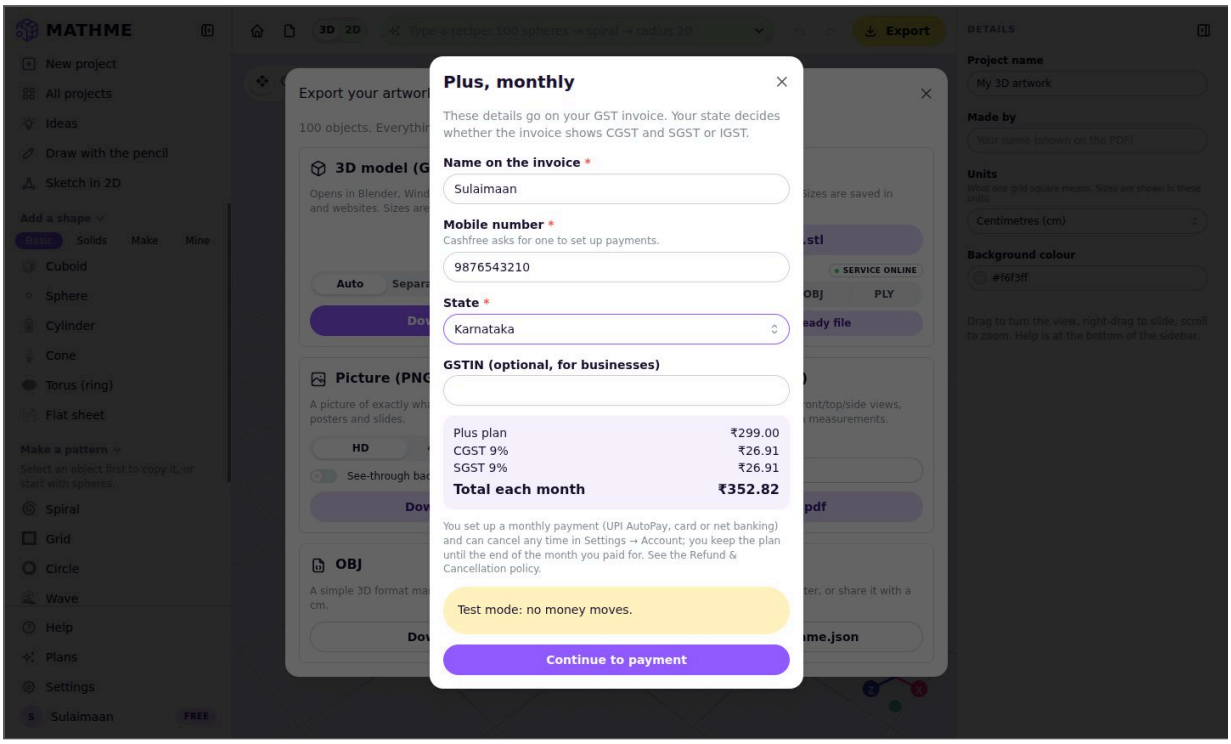


Figure 3.20: Checkout Page

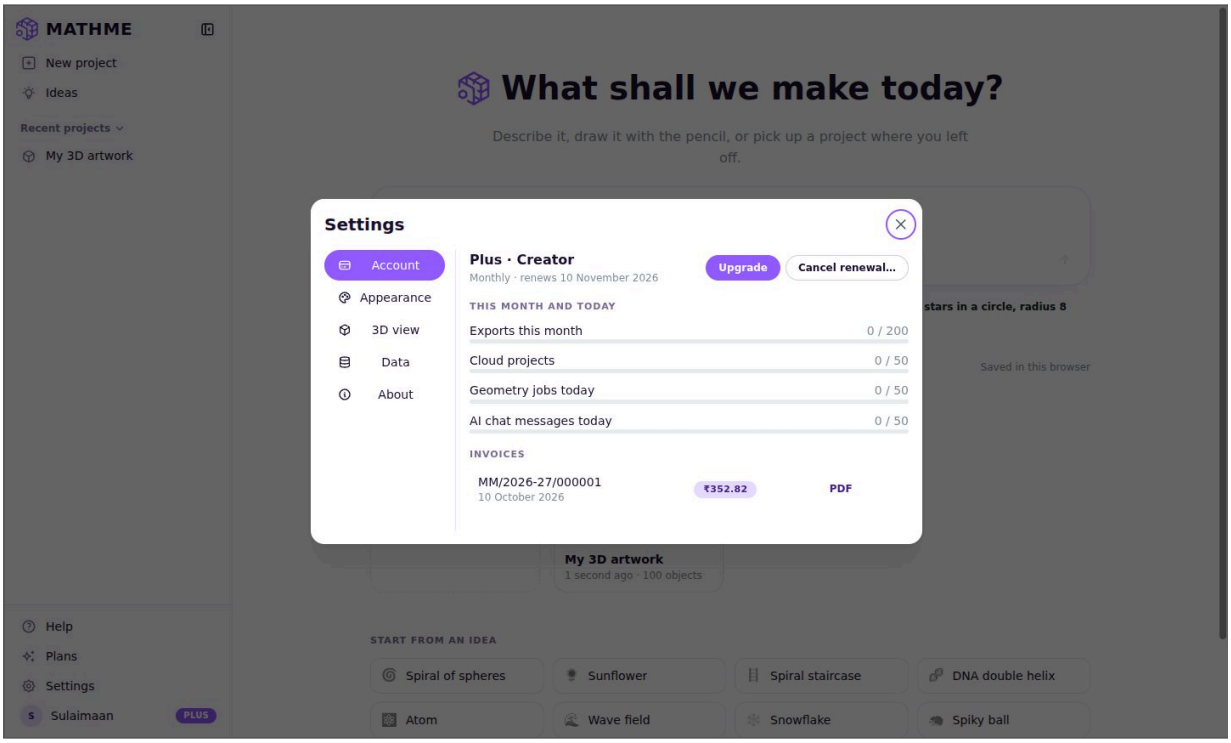


Figure 3.21: Account Settings Page

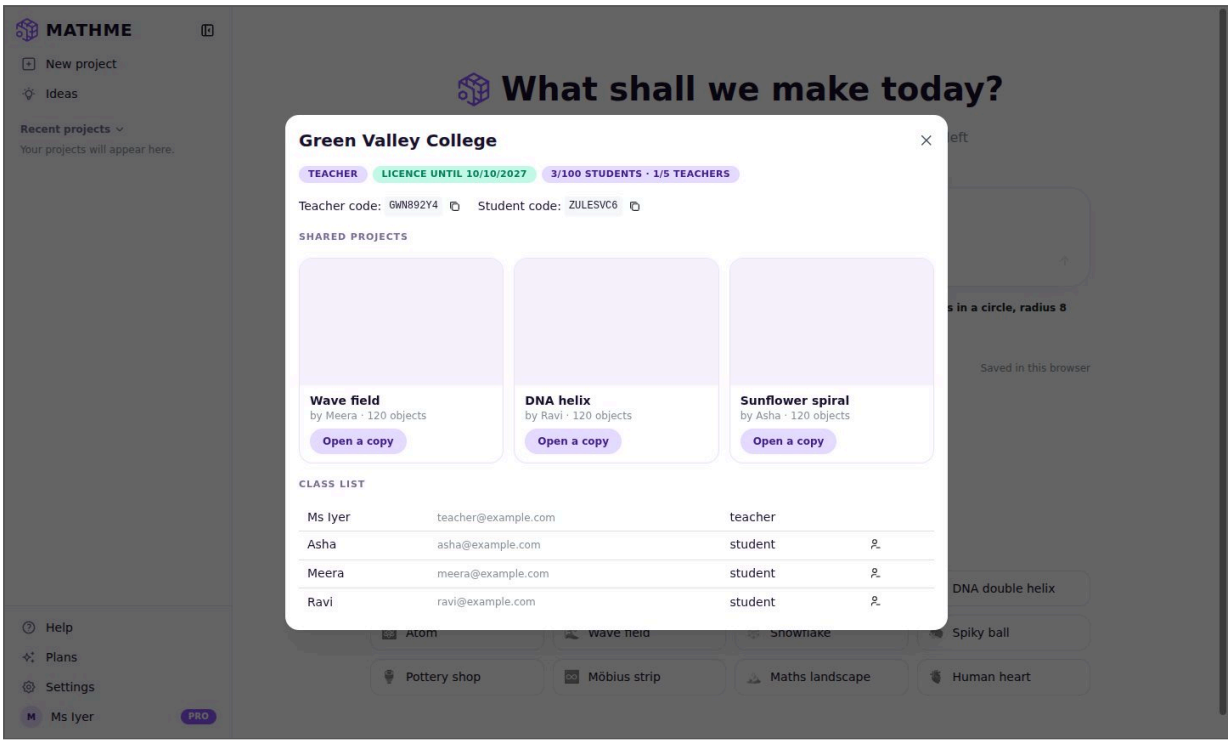


Figure 3.22: Class Page

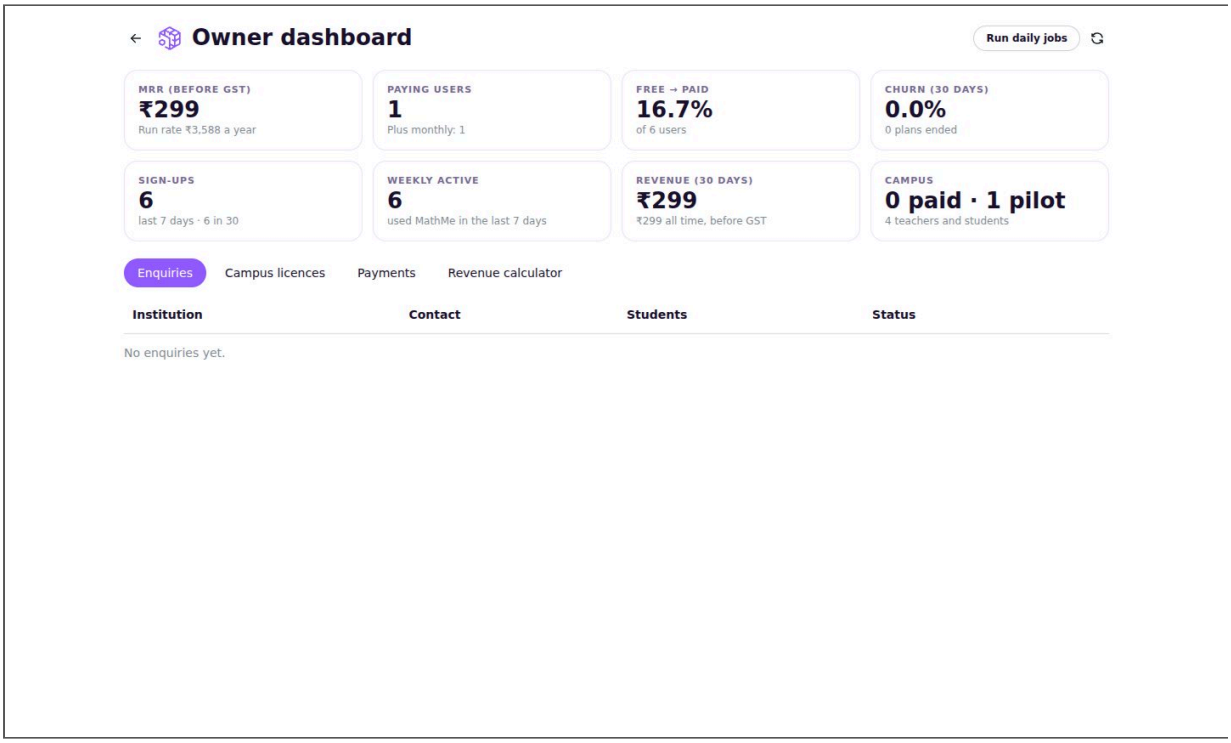


Figure 3.23: Owner Dashboard

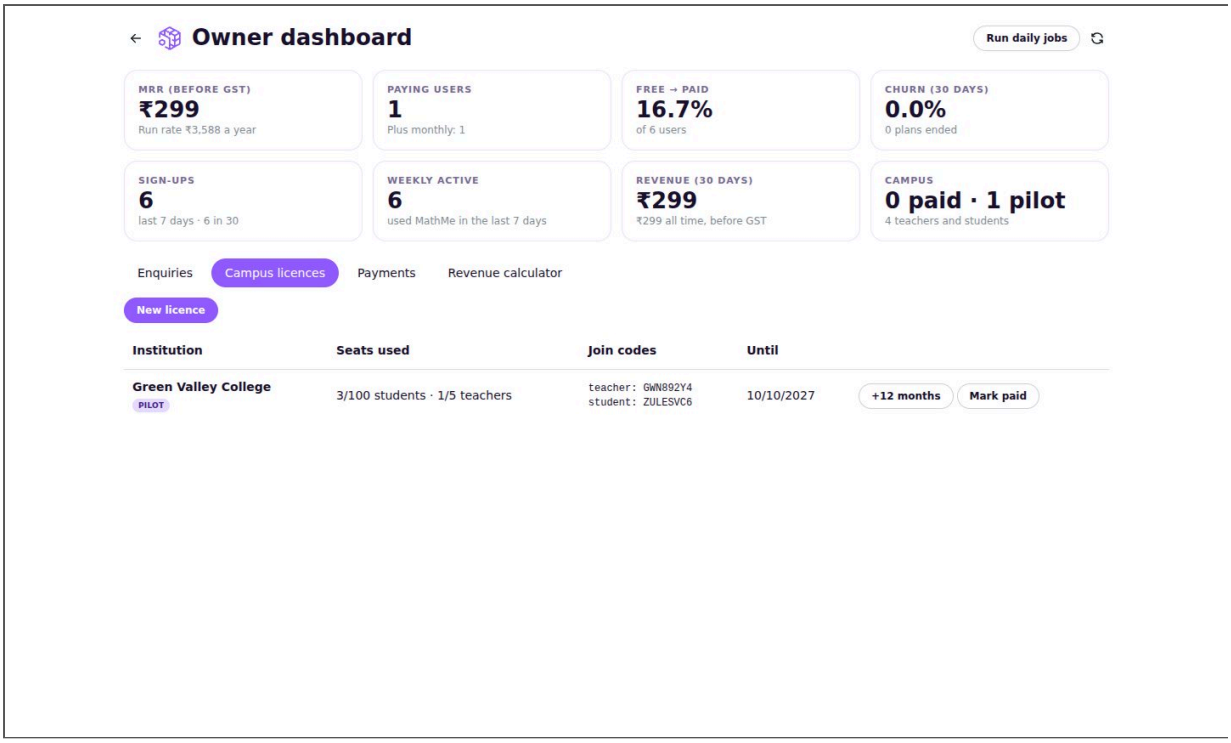


Figure 3.24: Campus Licences Page

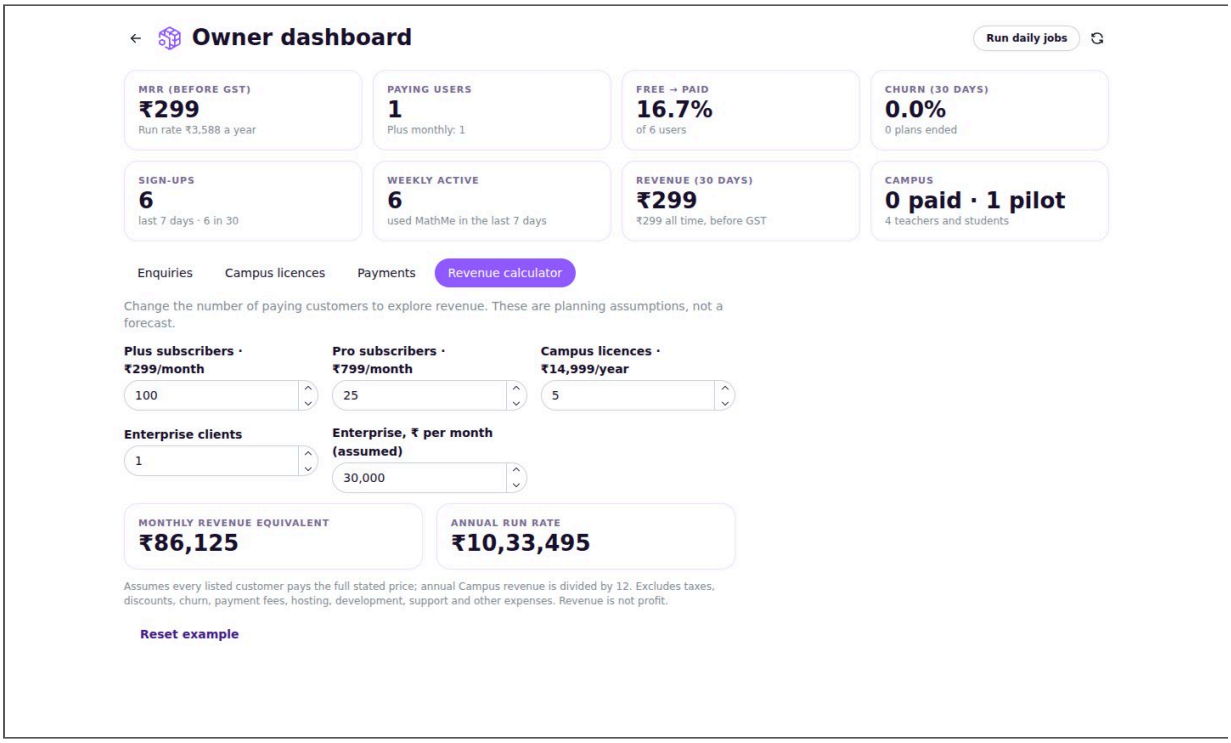


Figure 3.25: Revenue Calculator Page

3.6 Testing

3.6.1 Testing Strategy

Testing was done at four levels, and all of them run automatically in the continuous-integration pipeline:

- **Unit tests (Vitest)** check the maths engine: shapes, patterns, formulas, the recipe parser, the sketch solver, project files, plan limits and the revenue calculator.
- **Browser tests (Playwright)** open the real application in Chromium and act like a user: make a project, apply a pattern, draw with the pencil, sketch a triangle, export every file type, cut the heart, and so on.
- **Accounts browser tests** run the account service with a pretend payment provider and a test sign-in, and go through sign-in, limits, checkout with GST, cloud saves, the Campus flow, the dashboard and invoices.
- **Service tests (pytest)** check the geometry service (volumes, booleans, exports, the heart slicer) and the account service (sign-in, sessions, projects, quotas, GST, webhook signatures, subscriptions, Campus licences, admin access).

3.6.2 Test Cases

Table 3.8: Selected test cases

ID	Test scenario	Expected result	Result
TC1	Type “100 spheres → spiral → radius 20”	100 objects appear in a spiral	Pass
TC2	Select object 10 of the spiral, open Learn	Worked example shows θ = 300°	Pass
TC3	Sketch a 3-4-5 triangle and select it	Area 6 cm², perimeter 12 cm	Pass
TC4	Export GLB	Valid glTF 2.0 file, one mesh per object	Pass
TC5	Join two overlapping cuboids (service)	One watertight solid, correct volume	Pass

ID	Test scenario	Expected result	Result
TC6	Free user chooses STL export	Upgrade prompt names Plus	Pass
TC7	Checkout Plus from Kerala (seller in Karnataka)	IGST 18%, total ₹352.82	Pass
TC8	Webhook with a wrong signature or old timestamp	Rejected (401)	Pass
TC9	Teacher and student join with codes	Teacher gets Pro, student gets Plus	Pass
TC10	Sign-in link replayed in another browser	Sign-in refused (login CSRF guard)	Pass

3.6.3 Test Results

Table 3.9: Summary of automated test results

Test suite	Tool	Tests	Passed
Web application: unit tests	Vitest	147	147
Web application: browser tests	Playwright	29	29
Accounts: browser tests	Playwright	6	6
Account service	pytest	51	51
Geometry service	pytest	40	40
Total		**273**	**273**

In addition, the code passes ESLint, Prettier and the strict TypeScript compiler for the web application, and ruff (lint and format) for both Python services. The layouts were checked by hand on a desktop screen and at phone width.

3.7 Deployment Plan

The application was built so that it can be hosted cheaply and scaled easily. The web application and the account service are deployed together on Cloudflare: the Worker serves the website files and answers every request under /api. The geometry service runs as a Docker container on any container host. A step-by-step deployment guide (docs/DEPLOY.md) was written as part of the internship; Table 3.10 summarises it.

Table 3.10: Deployment steps

Step	Activity	Result
1	Create the D1 database and the R2 storage bucket; apply the SQL migrations	Tables ready in the cloud
2	Store the secrets (Google, Cashfree, Resend, geometry secret) with the Wrangler tool	No secret is kept in the code
3	Create a Google OAuth client with the sign-in callback address	Users can sign in with Google
4	Activate Cashfree, test in the sandbox, register the webhook address	Payments and webhooks work
5	Verify the email domain in Resend	Receipts and reminders are delivered
6	Run the geometry container with the shared secret	Measure, join and cut work for signed-in users
7	Build the web app with accounts on and deploy the Worker	MathMe is live on its own domain
8	Check a preview deployment end to end before real customers	A safe launch

Without accounts, the same web application can be published as a plain static website (for example on GitHub Pages or a school server), with every feature free and projects kept in the browser. This makes MathMe easy to try in any classroom.

3.8 Security Measures

Because the product handles accounts and payments, security was reviewed before the work was completed. The main measures are:

- Session cookies are **HttpOnly, Secure and SameSite=Lax**; only a SHA-256 hash of each session token is stored.
- Every request that changes data must carry a custom header that other websites cannot send, which protects against cross-site request forgery.
- The Google sign-in round trip is tied to the browser that started it with a short-lived state cookie, and the return address after sign-in is checked so that it cannot send users to

another website.

- Payment webhooks are accepted only with a valid HMAC signature, within a five-minute window, and only once per event.
- Test sign-in and the pretend payment page work only on a developer's own computer (localhost), so a forgotten setting cannot give anyone free plans or owner access.
- The geometry service can require a shared secret, so only the account service, which checks quotas first, can use it.
- All input is validated (Pydantic on the server, Zod for project files), and uploads are limited in size and processing time.

3.9 Learning During the Internship

The internship provided hands-on experience in many areas of computer applications:

- **Mathematics in software:** trigonometry, the golden angle, Fibonacci spheres, vectors, plane equations, area and volume formulas, and scaling laws (k^2 and k^3) turned into working code.
- **3D graphics:** meshes, materials, cameras, instanced rendering, coordinate systems and units, and file formats such as glTF, STL and OBJ.
- **Front-end development:** React components and hooks, state management with undo and redo, TypeScript types, responsive layouts and accessibility.
- **Back-end development:** REST APIs with FastAPI, input validation, SQL database design, migrations and serverless deployment on Cloudflare Workers.
- **Payments and compliance:** subscription mandates, webhooks, idempotency, GST calculation (CGST, SGST, IGST), invoice numbering per financial year, and the policy pages required by a payment gateway.
- **Security:** sessions and cookies, OAuth 2.0, CSRF protection, HMAC signatures and the principle of least privilege.
- **Software engineering:** planning in milestones, version control with Git, code reviews, automated testing at several levels, continuous integration and technical documentation.
- **Professional skills:** breaking a large goal into small steps, managing time across ten weeks, explaining technical work in simple language, and writing this report.

CHAPTER IV: INTERNSHIP OUTCOMES AND CONCLUSION

4.1 Internship Outcomes

The internship achieved all the objectives listed in Chapter I. Its main outcomes are:

- A **working web application**, MathMe 3D Studio, in which one mathematical rule creates hundreds or thousands of 3D objects, and every formula is shown and worked out for the user.
- A **maths engine** with 21 shape types, 7 patterns, variation and symmetry, a plain-language recipe parser and a 2D constraint solver, all covered by unit tests.
- A complete set of **exports** (GLB, STL, OBJ, PNG, PDF, SVG, DXF and project files), so that work can be 3D-printed, used in games and presented in class.
- A **Python geometry service** for exact measurements, solid booleans and print-ready files, and an educational **human-heart slice atlas** with 21 standard planes.
- A **2D sketch board** with measurements and geometric rules, and a **pencil** tool for drawing in 3D.
- An **account service** that makes the product sustainable: Google sign-in, cloud saves, plans in Indian rupees with GST-compliant invoices, Cashfree payments, Campus licences for schools and an owner dashboard.
- A **tested and documented** code base with continuous integration, a deployment guide and draft policy pages.

4.2 Achievements in Numbers

Table 4.1: The project in numbers

Measure	Value
Internship duration	90 hours over 10 weeks
TypeScript source files in the web application	138 files, about 18,400 lines
Python code in the two services	about 5,000 lines (plus about 1,200 lines of tests)
Shape types / patterns / ready-made ideas	21 / 7 / 12
Export formats	8 (GLB, STL, OBJ, PNG, PDF, SVG, DXF, project file)

Measure	Value
Heart atlas	51 structures, 21 standard cutting planes
Automated tests	273, all passing
Plans	Free, Plus, Pro, Campus and Enterprise
Database tables	14 (users, sessions, projects, subscriptions, payments, invoices, …)

4.3 Skills Gained

Table 4.2: Skills gained during the internship

Technical skills	Professional skills
TypeScript and React for large applications	Planning work in milestones and meeting them
3D graphics with Three.js and WebGL	Breaking a large problem into small, testable parts
Applying trigonometry and geometry in code	Writing clear documentation for different readers
Python, FastAPI and REST API design	Explaining technical ideas in simple language
SQL database design and migrations	Thinking about users: students, teachers and owners
Payment integration, webhooks and GST	Attention to legal and financial accuracy
Authentication, sessions and web security	Reviewing one’s own work critically
Automated testing (Vitest, Playwright, pytest)	Time management across a semester
Git, GitHub Actions and continuous integration	Presenting results with diagrams and screenshots

4.4 Challenges Faced and Solutions

Table 4.3: Challenges and how they were solved

Challenge	Solution
Thousands of objects made the 3D view slow	Instanced rendering: one shape and a list of positions per pattern
Formulas had to be correct and easy to read	Each pattern explains itself from the same numbers it uses to place objects, and tests compare both
Exact volumes of overlapping shapes are hard in the browser	A Python geometry service with trimesh and the manifold3d engine
Some hosts could not serve 3D model files	Load the models from a text (base64) copy when needed
Sketch rules can conflict with each other	An iterative solver that reports the rules it cannot satisfy instead of failing
Prices must match everywhere	One shared plans file read by the app, the server and the landing-page test
Money must never be counted twice	Signed webhooks, a table of processed event ids and amounts in whole paise
Login and payment safety	Security review: state cookie for sign-in, localhost-only test modes, shared secret for the geometry service
Testing payments without real money	A pretend payment provider that sends correctly signed webhooks in development

4.5 Limitations

- Some limits (export formats, the pattern list and 2D-to-3D) are checked in the browser and could be bypassed by a technical user; cloud saves, geometry jobs and AI messages are enforced on the server.
- The Cloudflare Worker adapters and the live Cashfree integration were tested with local substitutes and still need a test on the real services before launch.
- Real-time collaboration, class assignments and grading are not yet available.

- The heart atlas shows one reference heart and is meant for education only.
- Very large models are limited by the memory of the user's computer and by the size limits of the geometry service.

4.6 Future Enhancements

- Class assignments with deadlines, submissions and teacher feedback.
- Buying extra seats for a Campus licence, and changing plans with prorated prices.
- Real-time collaborative editing for group projects.
- More anatomy models (brain, skeleton) and more science topics.
- A marketplace where students and teachers share patterns and lessons.
- A public API for developers and offline support as an installable web app.
- Support for regional Indian languages in the interface.

4.7 Conclusion

The internship project "MathMe 3D Studio: Generative 3D Art & Object Studio" set out to show that mathematics can be the main tool of a creative 3D studio. The finished application lets a student write one rule, such as "100 spheres → spiral → radius 20 → rotation 30° → scale 0.5–2", and see every object placed by the formulas they study in class, with every step explained. The same tool measures, cuts, prints and exports real files, and it now has the accounts, plans, payments and school licences needed to be offered sustainably.

Building MathMe over ten weeks provided practical experience of the full software development life cycle, from requirement analysis and system design to implementation, testing, security and documentation, using current industry tools. It also showed the value of planning in small steps, testing everything automatically and writing for the people who will use the software.

The project is ready for a pilot with real users after its deployment on Cloudflare and the activation of the payment account. It forms a strong foundation for the future enhancements listed above.

BIBLIOGRAPHY

Books

1. Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th Edition, McGraw-Hill Education, 2020.
2. Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
3. Jos Dirksen, *Learn Three.js*, 4th Edition, Packt Publishing, 2023.
4. Alex Banks and Eve Porcello, *Learning React: Modern Patterns for Developing React Apps*, 2nd Edition, O'Reilly Media, 2020.
5. Boris Cherny, *Programming TypeScript*, O'Reilly Media, 2019.
6. Bill Lubanovic, *FastAPI: Modern Python Web Development*, O'Reilly Media, 2023.
7. Mark de Berg, Otfried Cheong, Marc van Kreveld and Mark Overmars, *Computational Geometry: Algorithms and Applications*, 3rd Edition, Springer, 2008.
8. Steve Marschner and Peter Shirley, *Fundamentals of Computer Graphics*, 5th Edition, CRC Press, 2021.
9. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019.
10. William Stallings, *Cryptography and Network Security: Principles and Practice*, 8th Edition, Pearson, 2020.

Online documentation

1. React documentation, https://react.dev
2. Three.js documentation, https://threejs.org/docs
3. Mantine UI documentation, https://mantine.dev
4. FastAPI documentation, https://fastapi.tiangolo.com
5. trimesh documentation, https://trimesh.org
6. Cloudflare Workers, D1 and R2 documentation, https://developers.cloudflare.com
7. Cashfree Payments developer documentation, https://www.cashfree.com/docs
8. Khronos Group, glTF 2.0 specification, https://registry.khronos.org/glTF
9. HuBMAP Consortium, Human Reference Atlas, https://humanatlas.io
10. Goods and Services Tax Council, GST law and invoice rules, https://gstcouncil.gov.in